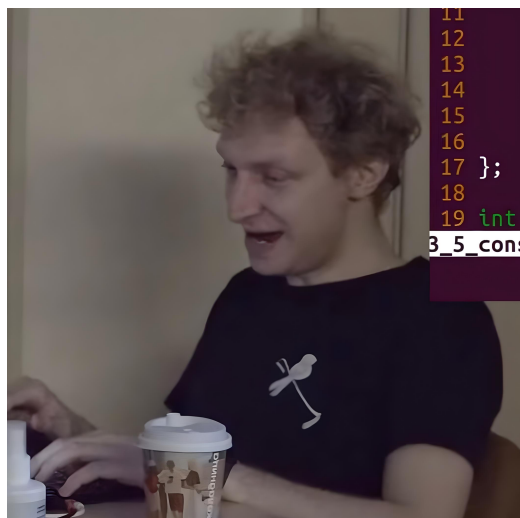


Московский физико-технический институт
Физтех-школа прикладной математики и информатики

Это Draft-версия

Б и л е т ы н а х о р
II СЕМЕСТР



«Я сейчас закричу..... ААААААА» (с) Илья Мещерин

Авторы:

*Бадоля Пётр,
Баронов Михаил,
Белецкий Иван,
Зинин Роман,
Золотуский Кирилл,
Савинов Даниил,
Столяр Дарья,
Сысоева Александра,
Шарилов Артем.*

лето 2024, компиляция от 26.06 00:45

Содержание

1 Компиляция программ	3
2 Стадии сборки	6
3 Пространства имен и поиск имен	8
4 Указатели, массивы и динамическая память	11
5 Приведения типов	13
6 Классы и структуры	14
7 Конструкторы и деструкторы	17
8 Перегрузка операторов	20
9 Константные и статические методы	22
10 Переопределение приведения типов	27
11 Перечисления (enum)	29
12 Видимость и доступность членов класса при наследовании	31
13 Публичное и приватное наследование	36
14 Множественное наследование	39
15 RTTI и dynamic_cast	42
16 Таблицы виртуальных функций	44
17 Специализация шаблонов	46
18 Специализации шаблонов	49
19 Простейшие compile-time вычисления	53
20 Простейшие метафункции	55
21 Шаблоны с переменным количеством аргументов (variadic templates)	58
22 Исключения	60
23 Исключения в конструкторах и деструкторах	66

24 Безопасность относительно исключений, спецификации исключений	70
25 Итераторы	72
26 Внутреннее устройство <code>std::vector</code>	78
27 Внутреннее устройство <code>std::deque</code>	82
28 Внутреннее устройство <code>std::list</code>	85
29 Внутреннее устройство <code>std::map</code>	87
30 Аллокаторы	91
31 Формальное определение <code>lvalue</code> и <code>rvalue</code>	93
32 Rvalue-ссылки	94
33 Проблема <code>perfect forwarding</code>	95
34 Ключевое слово <code>decltype</code> и конструкция <code>decltype(auto)</code>	96
35 Классы <code>unique_ptr</code> и <code>shared_ptr</code>	97
36 Лямбда-функции и замыкания	99
37 Шаблонное метапрограммирование и <code>SFINAE</code>	101
38 Ограничения, ключевое слово <code>requires</code>	103
39 Класс <code>std::function</code>	105

1 Компиляция программ

Разница между предупреждениями и ошибками компиляции. Параметры `-Werror`, `-Wall`, `-Wextra`. Примеры предупреждений, не являющихся ошибками. Версии компиляторов и версии языка. Компиляция в разных версиях языка. Оптимизации компилятора, уровни оптимизации. Примеры оптимизаций. Отличие UB от `unspecified behaviour`, пример последнего. Чем отличается приоритет операторов от порядка вычислений? Приведите примеры выражений с неоднозначным порядком вычислений.

Разница между предупреждениями и ошибками компиляции

Предупреждение: возможно, мы делаем что-то не то, но программа всё равно может скомпилироваться и отработать (возможно, некорректно) **Ошибка компиляции:** мы просто не можем скомпилировать программу, как бы ни хотелось.

Параметры `-Werror`, `-Wall`, `-Wextra`

`-Wall`: базовые предупреждения (33 штуки), но далеко не все. `-Wextra`: ещё 8 дополнительных предупреждений, вероятно, не таких необходимых. `-Werror`: а вот эта штука делает все предупреждения ошибками компиляции, т.е. при любом появившемся варнинге мы поймаем SE (причина неустойчивости вашего сына в школе).

Примеры предупреждений, не являющихся ошибками

(само собой, если не включён `-Werror`)

1. вызов функции "в никуда если она помечена как `[[nodiscard]]`;
2. орфографические ошибки, если (не дай бог) включена проверка;
3. невозможные (либо всегда верные) условия в `if/while/for`;
4. выходы за пределы массивов;
5. ещё какие-то UB, которые может отловить компилятор.

Версии компиляторов и версии языка

При запуске программы из терминала можно задать дополнительные параметры, например, `g++-14 -std=c++23 filename.cpp`.

`g++-14` — версия компилятора; как правило, выбирается последняя установленная версия данного релиза (подробнее [тут](#)). Обновления компилятора могут быть связаны не только с обновлениями стандарта (*поэтому не стоит искать связь между ними*), но и с хотфиксами и добавлением каких-либо оптимизаций, см. курс ТП.

`-std=c++23` — версия языка, обновляется раз в три года; да, стоит понимать, что старые версии компилятора вряд ли поддерживают новый стандарт языка.

Компиляция в разных версиях языка

Очевидно, что одна и та же программа в разных версиях может как компилироваться, так и нет; самый простой пример - банальное отсутствие какого-либо метода в `std` в той или иной версии языка (пусть будет `erase` в векторе, добавленный в C++20).

Иногда компиляторы это понимают, и, если вы пытаетесь запускаться в ненужной версии, кидают note с соответствующей пометкой.

Оптимизации компилятора, уровни оптимизации

Иногда компилятор пытается ускорить работу программы незаметно от нас. Пример такой оптимизации:

```

1 struct A {
2     virtual void f() {
3         std::cout << 1;
4     }
5 };
6
7 struct B : A {
8     void f() override {
9         new (this) A;
10        std::cout << 2;
11    }
12 };
13
14 int main() {
15     B b;
16     b.f(); // 2 (as expected)
17     b.f(); // 2 (1 expected)
18 }
```

Когда $f()$ вызывается впервые, b подменяется на объект типа A , но компилятор уже сохранил виртуальную функцию и не хочет лезть за ней заново, так как предполагает, что при втором вызове тип объекта останется тем же.

Конкретно эта оптимизация [чинится](#) но вообще оптимизации можно регулировать:

- ▷ $-O0$ — дефолтный вариант, включены некоторые оптимизации;
- ▷ $-O$ — оптимизации, не занимающие много времени в момент компиляции;
- ▷ $-O2$ — практически все поддерживаемые оптимизации;
- ▷ $O3$ — и ещё несколько вдовесок к предыдущему.

На самом деле флагов гораздо больше, про них можно почитать [тут](#).

Примеры оптимизаций

Вдобавок к примеру выше, есть, например, SSO (small string optimization) — если строка меньше какого-то фиксированного числа (условно 16 символов) — она хранится не в динамической памяти, а на стеке. Такие оптимизации есть не только в строках, но и, например, в function.

Самая, наверное, известная оптимизация — это RVO (return value optimization): если в функции возвращаемый объект создаётся в какой-то момент и после этого никак непосредственно не меняется, то компилятор не будет мувать (или даже копировать) его туда, где он должен в итоге оказаться, а сразу создаёт его там. Пример:

```

1 int pretty_good_sum(int a, int b) {
2
3     int c = a + b;
4     return c;
5 }
6
7 // This is actually the same as:
8
9 int probably_better_sum(int a, int b) {
10
```

```
11 return a + b;  
12 }
```

Отличие UB от unspecified behaviour, пример последнего

UB (undefined behaviour) — ситуация, когда, согласно стандарту, результат действия может быть любым - например, обращение к памяти, к которой формально нет доступа. Это плохо и этого мы всячески избегаем.

Unspecified behaviour — ситуация, когда стандарт не определяет поведение кода, и оно зависит от компилятора. Не является UB, но полагаться на такое нельзя и лучше этого избегать. Пример:

```
1 int f() {  
2     std::cout << "F" << std::endl;  
3     return 3;  
4 }  
5  
6 int g() {  
7     std::cout << "G" << std::endl;  
8     return 4;  
9 }  
10  
11 int h(int i, int j) {  
12     return i + j;  
13 }  
14  
15 int main() {  
16     return h(f(), g());  
17 }
```

Вывод программы может быть как FG, так и GF, в зависимости от того, в каком порядке мы вычисляем аргументы функции (а это определяется компилятором).

Чем отличается приоритет операторов от порядка вычислений?

- **Порядок вычислений**: вычисляемые значения не зависят друг от друга, и порядок влияет только на частные случаи (вроде примера выше);

- **Приоритет операторов**: устанавливается компилятором ещё на этапе синтаксического парсинга и определяет характер дальнейших вычислений (грубо говоря, расставляет скобки в нужном порядке). Как правило, это определено компилятором.

Приведите примеры выражений с неоднозначным порядком вычислений

Ну и ещё один пример:

```
1 int readValue()  
2 {  
3     int v;  
4     cin >> v;  
5     return v;  
6 }  
7  
8 int main()  
9 {  
10    cout << readValue() << ' ' << readValue() << '\n';  
11    return 0;  
12 }
```

Введённые значения могут быть выведены как в прямом, так и в обратном порядке.

2 Стадии сборки

Какие 4 стадии сборки существуют? Что происходит на каждой из стадий и что получается после каждой стадии? Пример ошибки линкера “undefined reference”. Как выполнить только отдельные стадии компиляции, не выполняя остальные?

Какие 4 стадии сборки существуют?

1. Препроцессинг
2. Компиляция
3. Ассемблирование
4. Линковка

Что происходит на каждой из стадий и что получается после каждой стадии?

Препроцессинг: обрабатываем директивы вроде `#include`, `#if`, `#pragma` и тому подобного. Например, при обработке «`#include`» мы вставляем в наш код указанный файл. (выходной файл: `name.cpp`, обработчик называется *препроцессором*).

Компиляция: превращаем наш файл в ассемблерный код (`name.s`). Компиляцией занимается, очевидно, **компилятор**.

Ассемблирование: собственно, `assembly` - сборка, и **ассемблер** занимается тем, что собирает инструкции из `name.s` в бинарник `name.o`. На выходе получается объектный файл.

Линковка: преобразуем объектный файл в исполняемый `name.out` (занимается этим **линкер**). Сейчас должно быть много вопросов, это нормально.

Важный момент: в препроцессинге в наш файл подставляются не готовые функции, а их определения, соответственно, на этапе ассемблирования там, где должны были лежать инструкции по их выполнению, лежат заглушки, а сами инструкции лежат в другом файле `.o`; задача линковщика - подставить в наш файл определение.

Зачем это? - Очевидно, чтобы не перекомпилировать всё каждый раз, потому что это займёт массу времени.

Пример ошибки линкера “undefined reference”

```
1 #include <iostream>
2
3 void some_great_function();
4
5 int main() {
6
7     some_great_function();
8 }
```

Функция не определена, и линковщик не понимает, с чем ему связывать `declaration`. Текст ошибки: `undefined reference to some_great_function()`.

Пример посложнее: мы пишем `#include library.h`, но при сборке в CMake не добавляем `library.cpp` в список исполняемых файлов. Проблема та же.

Как выполнить только отдельные стадии компиляции, не выполняя остальные?

- ▷ Препроцессинг: `g++ -E filename.cpp > new_filename.cpp`
- ▷ Компиляция: `g++ -S filename.cpp` (получаем `filename.s`)
- ▷ Ассемблирование: `g++ -c filename.cpp` (получаем `filename.o`)
- ▷ Линковка: `g++ filename.o`

3 Пространства имен и поиск имен

Примеры пространств имен. Анонимные пространства имен, их предназначение. Три вида объявления `using`, примеры. Пример затмения более локальным именем менее локального. Синтаксис обращения к глобальному имени. Понятие `qualified-id` и `unqualified-id`. Пример ошибки из-за неоднозначности при обращении к переменной.

Пространство имён - это удобный способ объявить область видимости и работать с ней. Из интересного - в неё можно в любой момент что-то дописать, т.е. допустимо такое:

```
1 namespace A {
2   int a = 5;
3 }
4
5 std::cout << A::a;
6
7 namespace A {
8   double b = a;
9 }
```

Объявлять пространства имён можно только внутри других пространств имён (в т.ч. `global namespace`).

Примеры пространств имен

Самый простой пример - `namespace std`, с другими мы в программе курса не сталкивались. Из интересного - в любое пространство имён можно что-то дописывать (например, когда мы хотим написать свою хеш-функцию в `unordered_map`).

Анонимные пространства имен, их предназначение

Рассмотрим вот такой код:

```
1 namespace
2 {
3     int i; // defines ::(unique)::i
4 }
5
6 void f()
7 {
8     i++; // increments ::(unique)::i
9 }
```

Я понятия не имею, что он значит, на лекциях этого не было, помогите

Три вида объявления `using`, примеры

1. **`using` для переменных:** позволяет использовать указанную переменную без указания неймспейса в той области видимости, в которой написан юзинг.

```
1 namespace N {
2
3   int room = 437;
4   int another_room = 305;
5 } // namespace N
6
7 N::room--;
```

```
8 using N::room; // now you can use it w/o N:: prefix
9 room++;
10
11 int main {
12
13     std::cout << room; // even in nested scope!
14
15     using N::another_room; // this one works for the scope where it's defined
16     another_room--; // like here
17 };
18
19 N::another_room++; // but not here
```

2. **using namespace** работает так же, т.е. добавляет в свою область видимости всё, что содержит соответствующее ему пространство имён:

```
1 namespace M {
2
3     std::string text = "You love funk, baby\n";
4
5     void print() {
6
7         using std::cout; // you can also do this if sure in safety
8         cout << func;
9     }
10
11 }
12
13 int some_function() {
14
15     using namespace M;
16
17     auto text2 = text;
18     print();
19 }
```

3. **using для типов (aliasing)**: создание псевдонимов для уже существующих типов. Чуть более умная штука, чем *typedef*.

Умеет в шаблоны, что, в частности и отличает от *typedef*:

```
1 template <typename T>
2 using v = std::vector<T>
3
4 int main() {
5
6     v<int> aboba{1, 2, 3};
7 }
```

Пример затмения более локальным именем менее локального

Объявление переменных с одинаковыми названиями и типами считаются redefinition только тогда, когда они объявлены в одной области видимости. Пример:

```
1 int main() {
2
3     int x = 0;
4
5     for (int i = 0; i != 1; ++i) {
6         int x = 1;
7         std::cout << x; // OK, output is 1
8     }
9 }
```

```

10  int x = 2;    // CE, now that's a redefinition
11  }

```

Замечание: можно объявлять вложенные области видимости без названия, и это будет корректно:

```

1  int x = 0;
2
3  {
4      int x = 1;
5      std::cout << x;    // OK, output is 1
6  }

```

Синтаксис обращения к глобальному имени

Мы всегда можем обратиться к глобальной переменной как переменной в пустом пространстве имён, т.е. `::x`. Например:

```

1  int x = 0;
2
3  {
4      int x = 1;
5
6      std::cout << x;    // OK, output is 1
7      std::cout << ::x;  // OK, output is 0
8  }

```

Понятие `qualified-id` и `unqualified-id`

unqualified-id - это последовательность символов, при помощи которой мы обозначаем название переменных, функций, классов и всего остального. Тут приведены правила построения. Также этим страшным словом называются кастомные операторы и штуки вида *ClassName* и *decltype(T)*.

qualified-id - это то, что я описал выше, но с префиксом *SomeName::*, где в качестве префикса выступает название класса или пространства имён.

Пример ошибки из-за неоднозначности при обращении к переменной

```

1  namespace A {
2
3      int x = 0;
4  }
5
6  int main() {
7
8      int x = 1;
9
10     using A::x;
11
12     std::cout << x;
13 }

```

Имеем: *error: 'int A::x' conflicts with a previous declaration*

4 Указатели, массивы и динамическая память

Чем `new` отличается от `malloc` и `calloc`? Чем `delete` отличается от `free`? Как `delete[]` узнаёт, сколько объектов нужно удалить? Что такое `variable length array (VLA)` и в чем их особенность? Чем отличается указатель на массив от массива указателей и от массива массивов, как всё это принимать в функции? Что такое указатель на функцию? Приведите пример объявления и использования указателя на функцию. Как работают указатели на функции при наличии перегрузки функций?

`new` — это `malloc/calloc` + конструктор объекта;

`delete` — это деструктор объекта + `free`;

На самом деле вместо `malloc` и `free` вызываются функции `void* operator new[](size_t bytes)` и `void operator delete[](void*)`, но, если их не изменять, то они делают то же самое, что и `malloc`, `free`.

Оператор `new[]` сохраняет информацию о количестве элементов массива перед началом массива. Оператор `delete[]` достаёт эту информацию и удаляет соответствующее количество элементов.

`Variable length array` (Массив переменной длины) представляет собой массив, длина которого определяется во время выполнения (а не во время компиляции).

```
1 int main() {
2     int n;
3     std::cin >> n;
4     int a[n];
5 }
```

Так делать не желательно, так как стандарт C++ запрещает так делать. (Хоть этот код и работает, но программа замедляется, так как в `runtime` она считает все отступы на стеке).

Особенностью VLA является возможность создания массивов, когда заранее неизвестно точное количество элементов.

`int a[5][5];` // массив из 5 массивов из `int`

`int* b[5];` // массив из 5 указателей на `int`

Информация справа от переменной приоритетнее, чем информация слева.

`int (*c)[5];` // указатель на массив из 5 элементов

Массив разрушается до указателей при передаче в функцию:

```
1 void f(int**) {}
2 void f(int(*)[5]) {} // f(int(*a)[5])
3 void f(int* [5]) {}
4 void f(int a[5][5]) {}
```

Первая и третья функции одинаковые (первая это указатель на указатель на `int`, а третья это массив из 5 указателей на `int`), а вторая и четвертая это указатель на массив из 5 `int`. При передаче в функцию массив воспринимается как указатель, но во втором примере мы передаем указатель на массив и в таком случае массив воспринимается как массив. В четвертом случае мы передаем массив, поэтому он воспринимается как указатель, но внутренний массив остается массивом.

У нас есть область памяти, в которой хранится исполняемый код. Указатель на функцию это указатель на начало области памяти, в которой записана реализация функции в машинном коде (находится в области `text`).

```
1 bool cmp(int x, int y) {
2     return x > y;
3 }
4
5 int main() {
6     int a[5] = {5, 3, 8, 2, 9};
7     std::sort(a, a + 5, &cmp);
8 }
```

Реализована сортировка по убыванию. (cmp должна задавать функцию порядка) Какой тип имеет &cmp? bool (*p)(int, int) = &cmp;

(Читается: указатель на функцию, которая принимает 2 int и возвращает bool)

```
1 bool (*p)(int, int) = cmp; //the same because of Function to pointer
   conversion
2
3 void f(int) {}
4 void f(double) {}
5
6 int main() {
7     &f; // CE
8     void (*p)(int) = &f;
9     void (*p2)(double) = &f;
10 }
```

Это 2 разных указателя, т к перегрузка происходит не только, когда мы вызываем функцию, но и при взятии указателя на функцию;

```
1 void f(int);
2
3 int main() {
4     void (*p)(int) = &f; //Linker error due to lack of memory location where
   implementation is written (undefined reference)
5 }
```

5 Приведения типов

Операторы `reinterpret_cast`, `const_cast`, примеры корректного и некорректного использования. Разница между `static_cast`, C-style `cast` и вышеназванными операторами. Что может и чего не может каждый из кастов. Какие ошибки будут в случае неправильного использования каждого из кастов.

`reinterpret_cast` — смотрим на биты одного типа, как на биты другого типа (не позволяет снимать `const`, а другие переходы между типами разрешены)

```
1 int main() {
2     int x;
3     float& f = reinterpret_cast<float&>(x); // UB
4 }
```

`reinterpret_cast` делается только к указателю или ссылке, так как мы не создаем новый объект, а смотрим на старый объект по-новому. `x` и `f` это два названия одной ячейки памяти, просто обращаясь к ней по разным именам читаем разные типы.

Некорректное использование(почти всегда и при попытке снять `const`):

```
1 int main() {
2     int x;
3     float& f = reinterpret_cast<float&>(x);
4     f = 3.14;
5     std::cout << x;
6 }
```

Корректное использование:

Есть две структуры, в которых лежат по очереди переменные одинаковых типов (то есть побитово эти структуры хранят одни и те же данные), между этими структурами не определена конверсия. В таком случае `reinterpret_cast` может помочь избежать копирования и просто прочесть одну структуру как другую.

`const_cast` — снимаем константность из-под указателя или ссылки(может только перейти от константного типа к нему же, но без константности)

Некорректное использование(почти всегда и при попытке изменить тип):

```
1 const int x = 5;
2 int& y = const_cast<int&>(x); // UB
3 y = 6;
```

(скорее всего константа все равно не поменяется)

Корректное использование: Если есть функция, в которую объект принялся по константной ссылке, но мы знаем, что объект изначально был не константный, то делать к нему `const_cast` легально.

C-style `cast` (запрещен в кодстайле, но может все) пытается по очереди применить такие касты: `const_cast`, `static_cast`, `const_cast` + `static_cast`, `reinterpret_cast`, `const_cast` + `reinterpret_cast`, если все не получилось, то СЕ.

```
1 int main() {
2     double d = 3.14;
3     int x = (int) d;
4 }
```

Если у какого-то из кастов просим сделать конверсию, которую он не умеет делать, то будет СЕ.

6 Классы и структуры

Агрегатная инициализация структур, примеры. Designated initializers. Внутренние классы, обращение к ним извне. Локальные классы. Размещение структур в памяти, пример изменения размера структуры от перестановки полей. Принцип “сначала разрешение перегрузки, затем проверка доступа”.

Размещение структур в памяти, пример изменения размера структуры от перестановки полей. Для структуры гарантируется, что её поля лежат в памяти в том же порядке, в котором они объявлены. Размером структуры является сумма размеров её полей с точностью до выравнивания, т.е. размером следующей структуры будет 4:

```
1 struct S {
2     int x = 1;
3 };
```

Вот так, соответственно – 8:

```
1 struct S {
2     double x = 1;
3 };
```

И вот такой – 8:

```
1 struct S {
2     int x = 1;
3     int y = 1;
4 };
```

А вот такой уже 16:

```
1 struct S {
2     int x = 1;
3     double y = 2;
4 };
```

Это происходит потому что компилятор кладёт 8-байтные переменные на кратные 8 адреса, и значит 4 байта будут пропущены: Теперь посмотрим на вот такую структуру:

```
16 0x7ffc5bdf0670 0x7ffc5bdf0678
```

```
1 struct S {
2     double z = 16;
3     int t = 1;
4     double y = 16;
5     int x = 1;
6 };
```

Её размером будет 32, поскольку даблы лежат на кратных 8 адресах. А вот если мы переставим t и y, то компилятор положит t и x рядом, и размером следующей структуры будет 24 байта:

```
1 struct S {
2     double z = 16;
3     double y = 16;
4     int t = 1;
5     int x = 1;
6 };
```

Агрегатная инициализация структур, примеры. Агрегатной инициализацией структуры называется определение её через фигурные скобки, в которых мы передаём значения, которыми должны быть проинициализированы поля в порядке их расположения в структуре:

```
1 struct S {
2     int x = 1;
3     double y = 16;
4 };
5
6 int main() {
7     S s {2, 4.5}; // Aggregate initialization
8 }
```

Также она может быть записана вот таким образом:

```
1 struct S {
2     int x = 1;
3     double y = 16;
4 };
5
6 int main() {
7     S s = {2, 4.5}; // Aggregate initialization
8 }
```

Соответственно, если полей меньше, чем в структуре, то поля, значения которым не были переданы будут проинициализированы по умолчанию, если больше, то СЕ.

Но агрегатная инициализация не работает для структур, у которых есть нетривиальные конструкторы, нет private/protected полей (а ещё нет виртуальных методов, виртуальной базы и нет прямого private/protected родителя) (Агрегатная инициализация работает и для классов, если они удовлетворяют перечисленным условиям)

Designated initializers. Также, некоторые поля при aggregate initialisation можно пропустить, и объявлять вот так:

```
1 struct A { int x; int y; int z; };
2
3 A a{.y = 2, .x = 1}; // error; designator order does not match declaration
   order
4 A b{.x = 1, .z = 2}; // ok, b.y initialized to 0
```

Но, все поля структуры, определённые так, должны создаваться или aggregate initialization, или просто присваиваться (причём narrowing conversion's не разрешены).

Внутренние классы, обращение к ним извне. Локальные классы. Внутри класса можно объявить как класс, так и структуру, и обратиться к ним следующим образом:

```
1 struct S {
2     struct SS {
3         static inline char c = '\n';
4     };
5 };
6
7 int main() {
8     S::SS::c;
9 }
```

Локальные классы. Локальным классом называется структура/класс внутри функции, например вот так:

```
1 int main() {  
2     struct S {  
3         char c;  
4     };  
5 }
```

Имя локального класса доступно только внутри функции. Локальный класс не может иметь статических членов. Все его функции должны быть определены в его теле.

Принцип “сначала разрешение перегрузки, затем проверка доступа”. Заметим, что если мы обращаемся к методу класса/структуры, то у нас сначала проводится разрешение перегрузки, а уже потом – можем ли мы получить доступ к функции, проверка на модификаторы доступа у нас происходит в конце компиляции, уже после выбора. Т.е. вот такой пример не скомпилируется:

```
1 struct S {  
2     private:  
3         void f(int) {}  
4     public:  
5         void f(long long) {}  
6 };  
7  
8 int main() {  
9     int x = 1;  
10    S s;  
11    s.f(x);  
12 }
```

Несмотря на то, что у нас есть подходящий публичный вариант, выберется приватная версия – по правилам перегрузки, и будет СЕ.

7 Конструкторы и деструкторы

Делегирующие конструкторы, пример. Как явно вызвать конструктор данного типа на данной памяти и когда это бывает нужно? В каких ситуациях уместно явно вызвать деструктор у объекта? В чем заключается идиома copy-and-swap? Что такое `std::initializer_list`? Приведите пример конструктора от `std::initializer_list`.

Что такое `std::initializer_list`? Приведите пример конструктора от `std::initializer_list`. `std::initializer_list<T>` — это такой лёгкий объект, который обеспечивает доступ к массиву элементов типа `const T`. Он автоматически создаётся, когда

- ▷ список в фигурных скобках используется для инициализации объекта при помощи соответствующего конструктора;
- ▷ список в фигурных скобках используется как правый операнд для оператора присваивания/вызова функции, и соответствующий оператор/функция принимают `std::initializer_list`;
- ▷ список в фигурных скобках после авто в range-based for

Вот так можно следать конструктор от `initializer_list` для `String`:

```

1 class String {
2     char* arr = nullptr;
3     size_t sz = 0;
4     size_t cap = 0;
5
6 public:
7     String() = default;
8
9     String(size_t n, char c): arr(new char[n + 1]), sz(n), cap(n) {
10         std::fill(arr, arr + sz, c);
11         arr[sz] = '\0';
12     }
13
14     String(std::initializer_list<char> list)
15         : arr(new char[list.size()])
16         , sz(list.size())
17         , cap(sz + 1) {
18         std::copy(list.begin(), list.begin() + sz, arr);
19         arr[sz] = '\0';
20     }
21 };

```

Делегирующие конструкторы, пример. Для `String` удобно определить конструктор просто от `size_t — n`, который выделяет память на `n char`’ов. После этого можно удобно переписать имеющиеся конструкторы и использованием делегирующего конструктора:

```

1 class String {
2     char* arr = nullptr;
3     size_t sz = 0;
4     size_t cap = 0;
5
6     String(size_t n): arr(new char[n + 1]), sz(n), cap(n + 1) {
7         arr[sz] = '\0';

```

```

8     }
9
10    public:
11        String() = default;
12
13        String(size_t n, char c): String(n) {
14            std::fill(arr, arr + sz, c);
15        }
16
17        String(std::initializer_list<char> list): String(list.size()) {
18            std::copy(list.begin(), list.begin() + sz, arr);
19        }
20
21    };

```

В чем заключается идиома copy-and-swap? Идиома copy-and-swap заключается в копировании объекта в copy-assignment операторе, а после его swap'ом с this:

```

1    void swap(String& other) {
2        std::swap(arr, other.arr);
3        std::swap(sz, other.sz);
4        std::swap(cap, other.cap);
5    }
6
7    String& operator=(String other) {
8        other.swap(*this);
9        return *this;
10    }

```

Это убирает необходимость следить за присваиванием самому себе, и уменьшает копирасту за счёт лишнего копирования.

Как явно вызвать конструктор данного типа на данной памяти и когда это бывает нужно? Для явного вызова конструктора типа по pointers можно использовать placement new:

```

1    // within any block scope...
2    {
3        alignas(T) unsigned char buf[sizeof(T)];
4
5        T* tptr = new(buf) T; // Construct a "T" object, placing it directly
6                               // into your
7                               // pre-allocated storage at memory address "buf".
8        tptr->~T();           // You must **manually** call the object's
9                               // destructor
10                               // if its side effects is depended by the program.
11    }

```

Зачем это может быть нужно? Тип может просто не иметь конструктора по умолчанию, и конструироваться от какой-то неизвестной нам комбинации аргументов, поэтому, в, например, деке мы напрямую конструируем объект inplace. Ещё это может пригодиться, если нам надо, чтобы какой-то объект (указатель/маленький функтор) лежал где-нибудь в каком-то известном месте на стеке.

В каких ситуациях уместно явно вызвать деструктор у объекта? Мы вызываем деструктор каждого объекта ровно один раз, когда нам необходимо уничтожить объект

– когда мы выходим из области видимости; использование объекта после вызова его деструктора является UB и не регламентировано стандартном. Явно вызвать деструктор объекта может быть нужно при поддержке `strong exception safety guarantee` для того/иного метода (например, в `deque`/`list`/`map`), для объектов созданных при помощи `placement new`.

8 Перегрузка операторов

Как правильно перегружать арифметические операторы, префиксный и постфиксный инкремент? Как правильно перегружать операторы сравнения? Как работает оператор `<=>` и как его определить для своих типов? Как делать перегрузку унарной звездочки и стрелочки? Какие операторы нельзя перегружать? Можно ли поменять приоритет или ассоциативность операторов?

Как правильно перегружать арифметические операторы, префиксный и постфиксный инкремент? Перегрузим операторы `+` и `+=` на примере комплексных чисел:

```
1 struct Complex {
2     double re = 0.0;
3     double im = 0.0;
4
5     Complex(double re): re(re) {}
6     Complex(double re, double im): re(re), im(im) {}
7
8     Complex& operator+=(const Complex& other) {
9         re += other.re;
10        im += other.im;
11        return *this;
12    }
13 };
14
15 Complex operator+(const Complex& first, const Complex& second) {
16     Complex copy = first;
17     copy += second;
18     return copy;
19 }
```

Почему мы делаем так, и выражаем `+` через `+=`? потому что при `+=` через `+` у нас получится очень неэффективная реализация, т.к. придётся делать много лишних копирований. И если для тривиальных классов это допустимо, то в любом сложном классе это повлечёт за собой сильную потерю производительности. + Вот в таком виде у нас работает RVO.

Также можно перегру

```
1 struct X
2 {
3     // prefix increment
4     X& operator++()
5     {
6         // actual increment takes place here
7         return *this; // return new value by reference
8     }
9
10    // postfix increment
11    X operator++(int)
12    {
13        X old = *this; // copy old value
14        operator++(); // prefix increment
15        return old;    // return old value
16    }
17
18    // prefix decrement
19    X& operator--()
```

```

20     {
21         // actual decrement takes place here
22         return *this; // return new value by reference
23     }
24
25     // postfix decrement
26     X operator--(int)
27     {
28         X old = *this; // copy old value
29         operator--(); // prefix decrement
30         return old;    // return old value
31     }
32 };

```

Как правильно перегружать операторы сравнения? Выражаем меньше, а остальные – через него:

```

1 bool operator<(const Complex& first, const Complex& second) {
2     return first.re < second.re || first.re == second.re && first.im < second.
    im;
3 }
4
5 bool operator>(const Complex& first, const Complex& second) {
6     return second < first;
7 }
8
9 bool operator<=(const Complex& first, const Complex& second) {
10    return !(second < first);
11 }
12
13 bool operator>=(const Complex& first, const Complex& second) {
14    return !(second > first);
15 }
16
17 bool operator==(const Complex& first, const Complex& second) {
18    return first <= second && first >= second;
19 }
20
21 bool operator!=(const Complex& first, const Complex& second) {
22    return first < second || first > second;
23 }

```

Как работает оператор $<=>$ и как его определить для своих типов?

Как делать перегрузку унарной звездочки и стрелочки?

Какие операторы нельзя перегружать? Можно ли поменять приоритет или ассоциативность операторов? Нельзя добавлять новые операторы, менять приоритет и ассоциативность операторов. Перегружать можно только следующие операторы:

op - any of the following operators: + * / % ^ & | ~ ! = < > + = - = * = / = % = ^ = & = | = << >> <<= >>= ! = <= >= <<> (since C++20) && || ++ -- , -> -> () []

- 1) overloaded operator;
- 2) user-defined conversion function;
- 3) allocation function;
- 4) deallocation function;
- 5) user-defined literal;

Остальные операторы мы перегружать не можем.

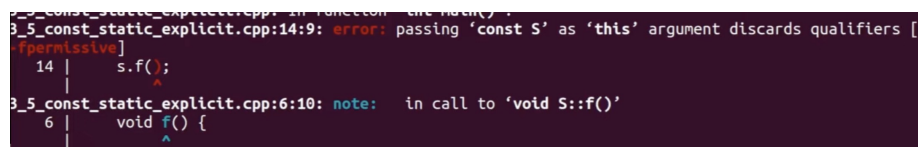
9 Константные и статические методы

Особенности поведения полей-указателей и полей-ссылок в константных методах. Ключевое слово `mutable` для полей и его применение. Особенности инициализации и размещения статических полей в памяти. Реализация класса `Singleton`. Статические константы, особенности их инициализации. Особенности инициализации статических полей, не являющихся константами.

Рассмотрим следующий код:

```
1 struct S {
2     void f() {
3         std::cout << "Hi";
4     }
5 };
6
7 int main() {
8     const S s;
9     s.f(); //CE
10 }
```

При попытке вызова метода `f` из `main` получаем СЕ.



Ровно, как и если бы мы попытались сделать так:

```
1 int main() {
2     S s;
3     const S& r = s;
4     r.f(); //CE
5 }
```

По умолчанию методы, которые мы определяем в классе, считаются не константными, то есть их нельзя вызывать у константных объектов (самое время вспомнить интуицию про константные объекты).

Происходит попытка сделать неявный каст от `const` к не `const` (метод `f` неявно принимает первым аргументом `this`), что запрещено.

Для того, чтобы метод был определен у константного объекта, нужно явно это прописать. Синтаксис следующий:

```
1 void f() const {
2     std::cout << "Hi";
3 }
```

Теперь вызов метода `f`, как в предыдущих примерах не будет СЕ.

То есть если мы хотим, чтобы метод можно было вызывать у не константного объекта, мы должны писать `const`. Пример: метод `size` у `string`.

Стоит обратить внимание, что в константных методах нельзя «менять поля», то есть вызывать у них не константные операции. Пример:

```
1 struct S {
2     int x = 0;
3     void f() const {
4         ++x; \\CE
5         std::cout << "Hi";
6     }
7 };
```

Рассмотрим следующий код:

```
1 struct S {
2     void f() const {
3         std::cout << 1;
4     }
5     void f() {
6         std::cout << 2;
7     }
8 };
```

Он корректен, то есть перегрузку по константности объекта можно делать (в случае вызова `f` от `const` объекта будет вызвана первая версия, в случае вызова `f` от не `const` — вторая).

Разумеется, к конструкторам и деструкторам это все отношения не имеет, к ним `const` писать нельзя.

Рассмотрим следующий код:

```
1 struct S {
2     char arr[10];
3
4     char& operator[](size_t index) const {
5         return arr[index];
6     }
7 };
```

Он скомпилируется и будет работать, но это по понятным причинам (не «запрещено» изменение для константного объекта) не совсем корректно. Правильнее будет сделать так:

```
1 struct S {
2     char arr[10];
3
4     const char& operator[](size_t index) const {
5         return arr[index];
6     }
7     char& operator[](size_t index) {
8         return arr[index];
9     }
10 };
```

Можно подумать «почему бы в случае `const` не вернуть просто `char`, присваивать же нельзя будет», но так делать нельзя, так как как минимум не будет работать взятие адреса от результата `[]`, еще рассмотрим следующий пример:

```
1 int main() {
2     String s = "abcd";
3     const String& cs = s;
4     const char& c = cs[0];
5     s[0] = 'b';
6     assert(c == 'b'); //not true if return type of operator[] is char, but
7 }                       must be the true
```

Поймем, что происходит в константных объектах с указателями и ссылками, если они были в полях. Если в классе полем был, например, `char* arr`, то в константном методе константным будет само поле, то есть `char* const arr` (полем является указатель, а не то, что под ним). К примеру, в рассмотренном ранее `struct S`, не получится инкрементировать `arr`, но если мы обращаемся к `arr` по индексу (`arr[index]`), то мы получим не константную ссылку на `char` (это компилируется).

Перейдём к случаю ссылок. Рассмотрим следующий код:

```
1 int x = 0;
2
3 struct S {
4     int& r = x;
5
6     void f() const {
7         ++r; //OK
8     }
9     void f(int y) const {
10        r = y; //OK
11    }
12 };
```

Это будет компилироваться. Внесём коррективы в интуицию о ссылках: подразумевается, что неизменными должны быть сами байты, которые кодируют ссылку, а не то, что будет, если ее разыменовать (ссылка в данном случае, по факту указатель), поэтому для ссылок нет разницы, в константном мы методе или нет.

Также следующий код тоже не является СЕ (но, вроде, это формальное UB):

```
1 struct S {
2     int x = 1;
3     int& r = x;
4
5     void f(int y) const {
6         r = y;
7     }
8 };
9
10 int main() {
11     S s;
12     s.f(1);
13 }
```

Иногда бывает так, что **в константном методе** хочется **поменять** поле, для этого используется `mutable`:

```
1 struct S {
2     mutable int x = 1;
3 };
```

ни при каких условиях `x` константным не будет. Это бывает полезно, например, для дебага (внутренние статистики) или метод `find` в `Splay`-дереве (можно реализовать `map` на нём). С точки зрения пользования это все еще `const`, но в нашей точки зрения — нет.

Также бывает необходимо, чтобы **метод относился не к объекту, а ко всему классу целиком**, в таком случае они называются статическими. Пример:

```
1 struct S {
2     static void f() {
3         std::cout << "Hi";
4     }
5 };
6
```

```

7 int main() {
8     S::f(); //example of using static function
9 }

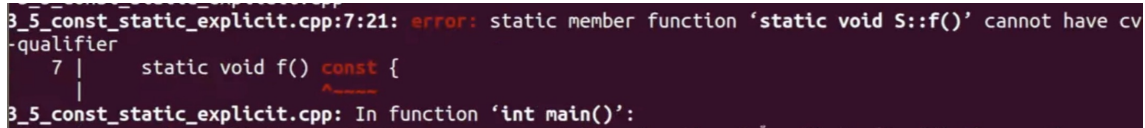
```

Стоит отметить, что `const` писать уже нельзя, то есть следующий код повлечет за собой СЕ, что логично:

```

1 struct S {
2     static void f() const {
3         std::cout << "Hi";
4     }
5 };

```



Также бывают *статические поля*. Это переменные, относящиеся не к конкретному объекту, а ко всему классу в целом. Они хранятся в статической памяти, класс — это как бы её namespace. Пример:

```

1 struct S {
2     static int x;
3 };

```

Стоит отметить, есть два «вида» статической памяти: которая инициализируется сразу и которая инициализируется при запуске. К примеру, если мы объявим глобальную статическую переменную и чем-то её проинициализируем (ещё и константную), то значение этой переменной прямо будет зашито в бинарник, или же если объявить глобальный статический массив «на очень много», то сам размер бинарника будет таким огромным, потому что этот массив будет записан туда по-байтово. А в случае статически не инициализированных переменных (которым мы изначально не придали значение) компилятор в бинарник запишет лишь число, равное сумме их размеров, а когда программа будет запускаться, под них под всех выделится статическая память и уже по мере того, как будет доходить дело до их инициализации, они будут инициализироваться.

Пример `static` в `BigInteger`: `base` (основание системы счисления) и `chunk` (количество бит/цифр в одном «чанке»).

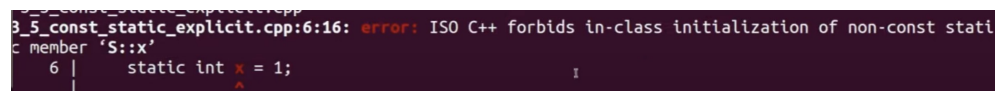
Со статическими полями есть одна странность:

```

1 struct S {
2     static int x = 1; //or any other number
3 };

```

Это будет СЕ



И правильно будет написать так:

```

1 struct S {
2     static int x;
3 };
4
5 int S::x = 1; //OK

```

Это было сделано для того, чтобы не было проблем с линковкой (класс можно `include`'ить много раз). В случае же `static const` можно:

```

1 struct S {
2     static const int x = 1; //OK
3 };

```

Классический пример применения статических методов и функций — класс Singleton.

Это класс, который может существовать лишь в единственном экземпляре. Реализация:

```
1 struct Singleton {
2     private:
3         Singleton() {}
4
5         static Singleton* ptr;
6
7         Singleton(const Singleton&) = delete;
8         Singleton& operator=(const Singleton&) = delete;
9
10    public:
11        static Singleton& getObject() {
12            if (ptr == nullptr) {
13                ptr = new Singleton();
14            }
15            return *ptr;
16        }
17 };
18
19 Singleton* Singleton::ptr = nullptr;
20
21 int main() {
22     Singleton& s = Singleton::getObject();
23 }
```

нужно ещё реализовать деструктор.

Наверное, стоит сказать что-то ещё про constexpr, но на лекции сказали об этом не сейчас:)

10 Переопределение приведения типов

Определение операторов приведения типа для класса (пример - конверсия `BigInteger` в `int`). Зачем нужно ключевое слово `explicit` и где его можно применять? Разница между `implicit conversion` и `contextual conversion`. Определение пользовательских литеральных суффиксов для класса (пример - суффикс `bi`).

В C++ есть ещё два ключевых слова, которые используются для объявления и определения методов — `explicit` (с английского *явный*), и «противоположное» ему — `implicit`. Это слово, которое применяется для методов, неявных вызовов которых мы хотим запретить. Такими методами являются 2 вещи: конструкторы и операторы приведения типа.

Поговорим сначала про операторы приведения типа.

Как известно, C++ язык со строгой типизацией, но, к примеру, иногда хочется, чтобы числа, несмотря на то, что являются `int`'ами, семантически имели другой тип. Это нужно, чтобы гарантировать, к примеру, передачу аргументов в функцию в правильном порядке (к примеру, если по факту это числа, но имеющие разный смысл; если их отдать в неправильном порядке, то все скопироваться, но упасть в Run time).

Любимый пример Ильи Мещерина с работы в яндекс такси (работал для души!!): в яндекс.такси, яндекс.картах есть такие типы — точка на карте (задается двумя координатами: широта и долгота). Внимание вопрос: что передается в функцию первым — широта или долгота? Не очевидно, а так можно оказаться не в Москве (55, 37), а в Иране (37, 55).

Замечание. Координаты указаны со слов Ильи Семировича и являются примерными.

Чтобы не перепутать, в данном случае можно завести 2 класса:

```
1 struct Latitude {
2     double value;
3     explicit Latitude(double value): value(value) {}
4 };
5
6 struct Longitude {
7     double value;
8     explicit Longitude(double value): value(value) {}
9 };
```

Мы хотим запретить неявные конверсии из них в `double` и из `double` в них (по умолчанию, это не так, в случае наличия конструктора), поэтому используем `explicit`.

Также C++ позволяет определять приведение типов. К примеру, из `Latitude` получить обратно `double`:

```
1 operator double() const {
2     return value;
3 }
```

Сейчас `operator double` написан так, что он может быть неявно вызван, то есть если какая-то функция принимает `double`, а мы передаем `Latitude`, то будет сделана неявная конверсия.

Начиная с C++11, операторы приведения типов могут тоже быть `explicit`, то есть

```
1 explicit operator double() const {
2     return value;
3 }
```

Теперь потребуется вызов `static_cast`, это как раз тот каст, который рассматривает, как стандартные конверсии, так и явно заданные нами, то есть фактически это: «Сделай определенную, известную тебе конверсию из одного типа в другой» (для стандартных типов они определены и так).

Вопрос на понимание: что будет, если написать так?

```
1 explicit operator double&() {
2     return value;
3 }
```

После такого `static_cast`'а (теперь он возможен) мы получим `value` (поле), которое теперь можно менять.

Считается, что конструкторы с одним аргументом лучше делать `explicit` (за редким исключением), как и операторы приведения типа.

Приведем ещё 2 примера:

1. пусть есть конструктор строки от `int` (строка с предвыделенным размером), его лучше делать `explicit`, так как

```
1 String s = 'a';
```

Конструктор от `int` не `explicit`, поэтому `'a'` сконвертируется в `int` и получится строка длины 97, заполненная «чем-то»;

2. в `BigInteger`, когда мы делаем конверсию в `bool` (чтобы использовать в `if`) его нужно делать `explicit`! (Это было сделано для решения проблемы неоднозначности) Для этого был сделан костыль под `if`'ом: `contextual conversion` (она есть под `while`'ом и тернарном операторе). Это особый вид конверсии, который разрешает в том числе рассматривать `explicit` конверсии.

Начиная с C++11, также можно переопределять ещё литеральные суффиксы (пример литерального суффикса — `2.0f`, то есть `float`). Делается это следующим образом (определением оператора):

```
1 class BigInteger {.....};
2
3 BigInteger operator""_bi(unsigned long long x) {
4     return BigInteger();
5 }
6
7 int main() {
8     BigInteger bi = 1_bi; //OK
9 }
```

Также возможно от `const char*`, но не от, к примеру, `int` (нужно брать максимально длинный числовой литерал).

Существует литеральный суффикс строки:

```
1 int main() {
2     "peresdacha"s; //std::string
3 }
```

11 Перечисления (enums)

Синтаксис определения enumов. Разница между enum и enum class. Мотивировка использования enumов. Указание нижележащего типа для enumа. Операции над enumами.

Бывает нужно, чтобы был тип, имеющий несколько фиксированных значений, причем эти значения несут какой-то конкретный смысл, называется enum. Пример:

```
1 enum E {
2     White,
3     Gray,
4     Black
5 };
6
7 int main() {
8     E e = White;
9 }
```

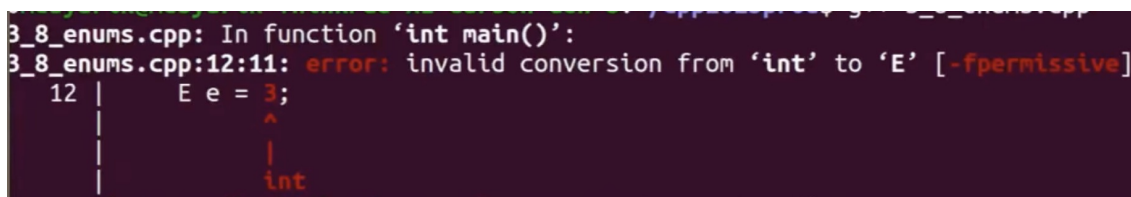
Это как бы именованные константы. На самом деле enum хранится в памяти как int. Стоит отметить, что можно написать и так:

```
1 int main() {
2     int e = White;
3     std::cout << e;
4 }
```

То есть по сути это эквивалентно тому, как если бы мы объявили 3 глобальных именованных константы (White = 0, Gray = 1, Black = 2).

Но обратно конвертировать нельзя, то есть:

```
1 int main() {
2     E e = 3; //CE
3 }
```



На самом деле это устаревшее и с C++11 появились enum классы:

```
1 enum class E {
2     White,
3     Gray,
4     Black
5 };
6
7 int main() {
8     E e1 = E::White; //now we must use it like this
9     E e2 = 1; //CE
10 }
```

Оно лишено недостатков enum'a, то есть она не вносит эти константы в глобальную область видимости и запрещает неявные конверсии (static_cast же разрешен) в обе стороны.

Когда есть какая-то переменная, которая может принимать всего ограниченный набор значений (и у них predetermined смысл) — стоит задуматься о enum'e.

Также можно явно писать, какие значения констант мы хотим, то есть

```
1 enum class E {  
2     White = 2,  
3     Gray = 3,  
4     Black = 9  
5 };
```

Также можно указать только первую (следующие автоматически будут +1 от предыдущей).

Также можно указывать каким типом мы хотим представлять enum, то есть

```
1 enum class E : int8_t {  
2     White,  
3     Gray,  
4     Black  
5 };
```

обязательно в целочисленном. обязательно в целочисленном.

12 Видимость и доступность членов класса при наследовании

Поиск имен при наследовании, сокрытие имен наследником. Явный вызов методов родителя у наследника. Использование `::` и `using`. Разница между видимостью и доступностью. Принцип “сначала поиск имен, затем проверка доступа”. Наследование конструкторов. Использование списков инициализации при наследовании, явная инициализация родителя в наследнике.

Рассмотрим такой код. Здесь «перекрывается» функция `f`.

```

1  struct Base {
2      void f() {
3          std::cout << 1;
4      }
5  };
6
7  struct Derived : Base {
8      void f() {
9          std::cout << 2;
10     }
11 };
12
13 int main() {
14     Derived d;
15     d.f(); // out: 2
16 }
```

В данной ситуации мы продолжаем руководствоваться мыслью «Частное предпочтительнее общего». Поэтому от объекта типа `Derived` вызовется метод класса `Derived`.

```

1  struct Base {
2      void f(int) {
3          std::cout << 1;
4      }
5  };
6
7  struct Derived : Base{
8      void f(double) {
9          std::cout << 2;
10     }
11 };
12
13 int main() {
14     Derived d;
15     d.f(0); // out: 2
16 }
```

В этом случае `Derived` рассматривается как локальная область видимости по отношению к `Base` и происходит «затмение» имен. Поэтому выберется `f(double)`

При этом, если мы в `Derived` объявим не `f(double)`, а `f()`, то будет СЕ, так как компилятор не найдет нужной функции для вызова. Объявление функции в наследнике с тем же именем **не является перегрузкой**.

Однако мы можем явно вызвать версию родительского класса:

```
1 struct Base {
2     void f(int) {
3         std::cout << 1;
4     }
5 };
6
7 struct Derived : Base{
8     void f(int) {
9         std::cout << 2;
10    }
11 };
12
13 int main() {
14     Derived d;
15     d.Base::f(0); // out: 1
16 }
```

При компиляции выбор версии происходит в таком порядке: набираем кандидатов из области видимости (то есть методы класса, от которого мы вызываемся), выбираем наиболее подходящего кандидата для перегрузки, проверяем приватность.

Мы можем явно внести методы родителя в ребенка, чтобы они наравне участвовали в перегрузке функций. При этом, если есть две идеально подходящие функции, но одна из Base, а другая из Derived, то выберется функция из Derived.

```
1 struct Base {
2     void f(int) {
3         std::cout << 1;
4     }
5
6     void g(int) {
7         std::cout << 3;
8     }
9 };
10
11 struct Derived : Base{
12     using Base::f;
13     using Base::g;
14     void f(double) {
15         std::cout << 2;
16     }
17
18     void g(int) {
19         std::cout << 4;
20     }
21 };
22
23 int main() {
24     Derived d;
25     d.f(0); // out: 1
26     d.g(0); // out: 4
27 }
```

Также `using`'ом мы можем изменить уровень доступа к функции. Пример:

```

1  struct Base {
2  protected:
3      void f(int) {
4          std::cout << 1;
5      }
6  };
7
8  struct Derived : Base{
9      void f(double) {
10         std::cout << 2;
11     }
12 };
13
14 int main() {
15     Derived d;
16     d.f(0); // out: 1, despite f(int) is protected
17 }

```

Аналогично с методами, члены класса затмевают друг друга. При этом если сделать `using` переменной, то будет СЕ (при совпадении имен).

```

1  struct Base {
2      int x = 1;
3      int y = 1;
4  };
5
6  struct Derived : Base{
7      using Base::y;
8      int x = 2;
9      int y = 2;
10 };
11
12 int main() {
13     Derived d;
14     std::cout << d.x; \\ out : 2;
15     std::cout << d.Base::x; \\ out : 1;
16     std::cout << d.y; \\ CE
17 }

```

Кринж пример с лекции:

```

1  struct Granny {};
2
3  struct Mom : private Granny {};
4
5  struct Son : Mom {
6      void f(::Granny& g) { // OK, parse as global name Granny
7          std::cout << g.x;
8      }
9      void f(Granny& g) { // CE, Son sees Granny as private parent and
10         // doesn't have access to this name
11         std::cout << g.x;
12     }
13 }
14
15 int main() {
16     Derived d;
17     d.f(0); // out: 2
18 }

```

Перейдем к конструкторам. Как и с обычными функциями с одинаковыми названиями, конструкторы не наследуются. Пример:

```

1      struct Base {
2          int x;
3          Base(int x) : x(x) {}
4      };
5
6      struct Derived : Base {
7          double y;
8          Derived(double y) : y(y) {}
9      };
10
11     int main() {
12         Derived d; // CE, Derived doesn't have default constructor
13         Derived d1(3.14); // CE, Base doesn't have default constructor;
14     }

```

Когда создается объект наследника, сначала создается объект родителя. Только после этого вызывается конструктор наследника. Мы можем либо убрать конструктор от *int x* у Base, либо добавить конструктор по умолчанию и тогда у нас программа будет работать корректно во втором случае. Также, мы можем в конструкторе наследника проинициализировать родителя:

```

1      struct Base {
2          int x;
3          Base(int x) : x(x) {}
4      };
5
6      struct Derived : Base {
7          double y;
8          Derived(double y) : Base(0), y(y) {}
9      };
10
11     int main() {
12         Derived d1(3.14); // OK
13     }

```

Деструкторы работают в обратном порядке. Сначала отрабатывает деструктор наследника и удаляет все поля наследника. После этого вызывается деструктор родителя и удаляет все поля родителя.

```

1      struct Base {
2          Base() {
3              std::cout << "Base ";
4          }
5          ~Base() {
6              std::cout << "~Base ";
7          }
8      };
9
10     struct Derived : Base {
11         Derived() {
12             std::cout << "Derived ";
13         }
14         Derived() {
15             std::cout << "~Derived ";
16         }
17     };
18
19     int main() {

```

```
20     Derived d;  
21 }
```

Вывод программы будет таким: Base Derived ~ Derived ~ Base. В C++11 добавили наследование конструкторов. Мы можем явно сказать, что мы можем создавать наследника при помощи конструкторов родителя (чтобы не дублировать код).

```
1     struct Base {  
2         int x;  
3         Base(int x) : x(x) {}  
4     };  
5  
6     struct Derived : Base {  
7         using Base::Base;  
8     };  
9  
10    int main() {  
11        Derived d1(3); // OK  
12    }
```

Теперь мы можем создавать Derived от int, так как такой конструктор есть у Base. Конструкторы копирования не наследуются, потому что это будет приводить к неявному касту от наследника к конструктору.

13 Публичное и приватное наследование

Разница между наследованием классов и структур. Как меняется доступность членов родителя при приватном наследовании по сравнению с публичным? Наследуются ли друзья? Как работают неявное приведение типов, `static_cast`, `reinterpret_cast` при публичном и приватном наследовании?

Сейчас будет безудержное веселье. © Мещерин И.С.

Разница между наследованием классов и структур состоит в том, что наследование классов по умолчанию `private`, а у структур — `public`.

Изменение доступа к члену или функции класса у наследника может быть совершено, если наследование имело тип `protected` или `private`, то есть если член класса был `public`, то после таких наследований он станет соответственно `protected` или `private`. Рассмотрим пример:

```

1 struct Granny {
2     int x;
3     void f() { }
4 };
5
6 struct Mom : protected Granny {
7     int y;
8     void g() {
9         std::cout << x;
10    }
11 };
12
13 struct Son : Mom {
14     int z;
15     void h() { }
16 };

```

Далее будем считать, что мы все вызовы пытаем сделать из `main`, а также у нас заведен объект `Sons`;

1. `s.z` — всё хорошо, так как `z` — `public` член структуры `Son`.
2. `s.y` — тоже все хорошо. `y` — `public` член `Mom` и `Son` публично наследуется.
3. `s.x` — СЕ, так как `Mom` имеет `protected` наследование от `Granny`, соответственно для сына `x` также является `protected` членом `Son`, а значит для `main` `x` скрыт, однако «изнутри» `Son` доступ к `x` имеется.

При этом, даже если мы напишем «внутри» `Granny` `friend int main()`, это не повлияет на доступность `s.x` из `main`, так как тип доступ изменяет `Mom`, даже несмотря на то, что `Granny` разрешила доступ к `x` из `main`. Аналогично, если мы поменяем вид наследования `Mom` от `Granny` на `private`, строчка `friend struct Son` не изменит характер доступа к члену, так как доступ запрещает `Mom`. «Злая мама запрещает общаться с доброй бабушкой.»

Еще немного изменим пример:

```

1 struct Granny {
2     public:
3         friend int main();
4         friend struct Son;
5     private:
6         int x;
7         void f() { }
8 };
9
10 struct Mom : private Granny {
11     int y;
12     void g() {
13         std::cout << x;
14     }
15 };
16
17 struct Son : Mom {
18     int z;
19     void h() { }
20 };

```

В данном примере доступа к x нет ни у Son, потому что доступ запретила Mom, несмотря на то, что Son — friend для Granny. В то же время, у Mom нет доступа к x , так как x — приватное поле Granny. Из этого примера можно понять, что friend'ы не наследуются.

Также, важно сказать, что все вышеописанные правила распространяются только на свои поля (например, поля Granny внутри Son как поля класса, от которого было наследование), но если в метод класса будет передан другой объект, то доступа не будет.

Поговорим про явные касты теперь. При наследовании мы говорим, что ребенок — частный случай родителя. Ключевая идея наследования в том, что такое действие — корректное.

```

1     struct Base {
2         int x = 1;
3     };
4     struct Derived : Base {
5         int y = 2;
6     };
7
8     void f(Base& b) {
9         std::cout << b.x;
10    }
11
12    int main() {
13        Derived d;
14        f(d);
15    }

```

При этом, при передаче Derived в f, происходит **неявный каст**. При этом, обратное действие — невозможно (привести Base к Derived). Мы также можем передавать не по ссылке, а по значению. При этом происходит slicing, то есть часть от Derived обрезается и по переданному объекту создается объект типа Base. Соответственно, мы не можем «расширить» Base до Derived и мы не можем передавать Base по значению туда, где ожидается Derived. Аналогично, мы можем передавать указатель на Derived туда, где ожидается Base.

Обобщим знания об implicit cast'ах и добавим static_cast и reinterpret_cast.

	Derived to Base			Base to Derived		
	copy	ptr	ref	copy	ptr	ref
implicit	OK	OK	OK	CE	CE	CE
static	OK	OK	OK	CE	OK	OK
reinterpret	CE	OK	OK	CE	OK	OK

Важно заметить, что хоть при reinterpret_cast и static_cast указателя и ссылки Base к Derived не происходит CE, вполне вероятно **может произойти UB**.

При этом, все вышеописанное корректно только для public наследования. В случае private или protected наследования, все ОК заменяются на CE, кроме reinterpret_cast.

14 Множественное наследование

Неоднозначности при множественном наследовании. Размещение объектов в памяти при множественном наследовании. Пример, когда от приведения типов меняется численное значение указателя. Что такое проблема ромбовидного наследования? В чем особенность обращения к прародителю при ромбовидном наследовании? Как работает `static_cast`, `reinterpret_cast` при множественном наследовании, при ромбовидном наследовании (в том числе когда присутствует приватное наследование)?

Главная проблема при множественном наследовании — несколько раз отнаследоваться от одного и того же через разных родителей.

```
1      struct Mom {
2          int m;
3      };
4
5      struct Dad {
6          int d;
7      };
8
9      struct Son : public Mom, public Dad {
10         int s;
11     };
```

Для каждого класса, от которого мы наследуемся должен быть прописан тип наследования. Если он опускается, то используется тот, что по умолчанию: для классов — `private`, для структур — `public`.

В памяти они будут лежать в следующем порядке: Mom(8 byte), Dad(8 byte), Son(8byte), то есть в том порядке, в котором мы перечисляли их при объявлении класса.

Возникает вопрос, как нам передавать Son& по ссылке, где ожидается Dad&? Это работает, как и работало с не множественным наследованием: мы можем передавать ребенка по ссылке или указателю на родителя. При этом указатель на объект смещается, так как (если рассматривать данный пример) «начало» Dad лежит не там же, где «начало» Son. То есть при таком действии: $Dad^*p = \&s$, где s — объект сына, сдвигается указатель на объект.

*Самое интересное происходит, если у нас есть мама, папа, но они siblings. ©Легенда
си плас плас*

Перейдем к ромбовидному наследованию (diamond problem). Это проблема, причем не только в жизни, но и в нашем любимом языке.

```

1      struct Granny {
2          int g;
3      };
4
5      struct Mom : Granny{
6          int m;
7      };
8
9      struct Dad : Granny {
10         int d;
11     };
12
13     struct Son : Mom, Dad {
14         int s;
15     };
16
17     int main() {
18         Son s;
19     };

```

Объект Son будет занимать 20 байт, потому что в нем два экземпляра Granny (от Mom и от Dad). Как он будет располагаться в памяти: Granny Mom Granny Dad Son.

Если мы попробуем обратиться к члену *g*, то будет ambiguous call, так как непонятно к которому *g* мы хотим обратиться. Мы можем явно обратиться к версии от одного из родителей *s.Mom :: g*, при этом *s.Granny :: g* будет CE.

При попытке сделать каст к Granny будет новая для нас ошибка ambiguous cast. При этом эта ошибка будет возникать и при slicing, и при касте к Granny& и при касте к Granny&.

```

1      int main() {
2          Son s;
3          static_cast<Granny>(s); \\ CE
4          static_cast<Mom::Granny>(s); \\ CE
5          static_cast<Granny>(static_cast<Mom>(s)); \\ OK
6      }

```

Добавим какой-нибудь метод в Granny. Если мы будем вызывать этот метод от объекта Son, то это тоже будет ambiguous call, так как метод может использовать поля Granny.

```

1      int main() {
2          Son s;
3          Mom& m = s; \\ OK
4          Dad& d = s; \\ OK
5          md = static_cast<Mom&>(d); \\ CE
6          dm = static_cast<Dad&>(m); \\ CE
7      }

```

Несмотря на то, что под ссылками *m* и *d* у нас лежит Son, мы не можем делать горизонтальный каст, потому что так решил комитет по стандартизации.

Заберем отца из семьи. Посмотрим на такой случай:

```

1 struct Granny {
2     int g;
3 };
4
5 struct Mom {
6     int m;
7 };
8
9 struct Son : Granny, Mom {
10     int s;
11 };

```

Мы сталкиваемся с теми же проблемами. У нас снова две Granny, которые равнозначны. Мы получили ситуацию, когда мы не можем явно обратиться к Granny, от которой мы наследуемся. Мы можем либо явно обратиться к ней через Mom, либо через `reinterpret_cast` со сдвигом указателей.

Разберем случай private наследования:

```

1 struct Granny {
2     int g;
3 };
4
5 struct Mom : private Granny {
6     int m;
7 };
8
9 struct Dad : private Granny {
10     int d;
11 };
12
13 struct Son : Mom, Dad {
14     int s;
15 };
16
17 int main() {
18     Son s;
19     static_cast<Granny*>(s); // ambiguous call
20     static_cast<Granny*>(static_cast<Mom*>(s)); // Granny is private parent
        for Mom
21 }

```

Как мы видим, если мы не объявляем friend'ом main в Mom или Dad, то `static_cast` к Granny через этого родителя невозможен. При этом, `reinterpret_cast` возможен. (Но, наверное, будет UB, я не уверена, а в лекциях этого не было). Однако строчка `reinterpret_cast<Granny*>(reinterpret_cast<Mom*>(s))` не вызовет СЕ, как и строчка `reinterpret_cast<Granny*>(s)`. Однако это справедливо только для ссылок и указателей.

15 RTTI и `dynamic_cast`

Пример, когда на этапе компиляции невозможно понять, какую версию функции придется вызвать. Понятие RTTI. Оператор `dynamic_cast` и его принцип действия. Пример ситуации, когда все три каста - `static`, `dynamic`, `reinterpret` - дают разный результат. Оператор `typeid` и его действие. Класс `typeid`, проверка типов объектов на равенство, вывод названия типа на экран.

При использовании полиморфных классов, может быть такое, что на этапе компиляции мы можем не знать, объект какого типа лежит под ссылкой. Приведем пример такого кода:

```

1      struct Base {
2          int x;
3          virtual void f() {
4              std::cout << "Base";
5          }
6      }
7
8      struct Derived : Base {
9          int y = 0;
10         void f() override {
11             std::cout << "Derived";
12         }
13     }
14
15     void g(Base& b) {
16         b.f();
17     }
18
19     int main() {
20         Base b1;
21         Derived d;
22         g(b1); \\ out : Base
23         Base& b2 = d;
24         g(b2); \\ out : Derived
25     }
```

В обоих случаях мы передаем в функцию объект `Base&`, но в первом случае там лежит `Base`, а во втором `Derived`. То есть передавая объект `Base`, мы не можем быть уверены в compile time, что там будет выведено и какая версия выберется. Однако у нас есть несколько способов понять в runtime'e какой объект на самом деле лежит.

`dynamic_cast` — каст, который делается в runtime. Соответственно, во время компиляции мы не можем понять совершится он или нет. Если мы делаем каст к указателю, то в случае неудачного каста, нам вернется `nullptr`, а если к ссылке, то бросится исключение `std::bad_cast`.

```

1      struct Base {
2          int x;
3          virtual void f() {}
4          virtual ~Base() = default;
5      }
6
7      struct Derived : Base {
8          int y = 0;
9          void f() override {}
10     }
```

```

11
12     int main() {
13         Derived d;
14         Base& b = d;
15         Derived* bd = dynamic_cast<Derived*>(&b);
16
17     }

```

RTTI — RunTime Type Information. Мы можем явно спросить как называется тип, который лежит под ссылкой. Для этого существует оператор **typeid**. Он возвращает тип `std::typeinfo`, у которого есть два метода: оператор сравнения и `name()`, позволяющий узнать название типа.

```

1     int main() {
2         Derived d;
3         Base& b = d;
4         std::cout << typeid(b).name(); \\ 7Derived;
5         if (typeid(d) == typeid(b)) {
6             \\
7         }
8     }

```

7 — длина названия типа, а `Derived` — название типа.

От любого полиморфного типа можно сделать `dynamic_cast` к `void*`. Также, мы можем сделать `dynamic_cast` от наследника к родителю даже если типы не полиморфные (это полезно, так как если мы сделаем неверный каст со `static_cast`, то мы получим UB).

При множественном наследовании, мы можем сделать `dynamic_cast` вбок (от `Mom` к `Dad`), если наследник полиморфный.

Два примера, когда `static_cast`, `dynamic_cast` и `reinterpret_cast` дают разные результаты:

1. У нас виртуальный предок и мы кастим от предка к наследнику. `dynamic_cast` успешно отработает, `static_cast` дает СЕ, а `reinterpret_cast` дает UB
2. Каст вбок: `dynamic_cast` корректно обрабатывает, `static_cast` дает СЕ, а `reinterpret_cast` дает UB

16 Таблицы виртуальных функций

Размещение полиморфных объектов в памяти. Разница в представлении пустой структуры с виртуальными функциями и без таковых. Понятие таблицы виртуальных функций (vtable), общее представление о ее содержимом. Объяснение работы vtable на стандартном примере с одним родителем и одним наследником. Пример с двухуровневым наследованием, когда родитель верхнего уровня не полиморфный.

Все, что далее будет рассказано в билете, не является стандартом языка C++. Однако примерно во всех компиляторах эти вещи устроены одинаково.

Если в классе существует хотя бы одна виртуальная функция, то этот класс будет весить хотя бы 8 байт, так как внутри класса хранится указатель на **vtable**. **Виртуальная таблица** или **vtable** — структура данных, хранящаяся в статической памяти, в которой перечислены адреса виртуальных функций для этого типа. Для одного типа существует одна виртуальная таблица.

Посмотрим на такую структуру:

```
1 struct Base {
2     virtual void f() {}
3 };
```

Виртуальная таблица этого класса выглядит следующим образом:

| &Base typeid | &Base::f |

Первая ячейка — указатель на статическую память, где лежит объект типа std::typeid про данный класс. Вторая ячейка хранит указатель на адрес функции Base::f. При этом в самой структуре указатель указывает на начало &Base::f.

Добавим наследника:

```
1 struct Base {
2     virtual void f() {}
3     void h() {}
4     int x;
5 };
6
7 struct Derived : Base {
8     void f() override {}
9     void g() {}
10    int y;
11 };
```

Как в памяти будет выглядеть Base и Derived?

Base: | ptr | x | padding |

Derived: | ptr | x | y |

Опишем vtable для Derived:

| &Derived typeid | &Derived::f | &Derived::g |

При вызове виртуальной функции, мы идем по указателю на vtable. При вызове, мы знаем какой номер у вызываемой виртуальной функции (по названию). Сдвигаемся на нужное количество байт туда, где у нас лежит указатель на адрес функции.

Если у нас по ссылке на родителя лежит объект ребенка, то мы пойдем по указателю на виртуальную таблицу для ребенка и вызываем нужную функцию.

Рассмотрим такой пример:

```

1  struct Granny {
2      int g;
3      void h() {}
4  }
5
6  struct Mom : Granny {
7      virtual void f() {}
8      int m;
9  };
10
11 struct Son : Mom {
12     void f() override {}
13     int s;
14 };

```

Тут Granny — не полиморфная. Как будет выглядеть объект Son:

| ptr | g | m | s |

Если мы делаем каст от Son к Granny, то у нас указатель у нас указатель на начало объекта сдвигается на начало *g*, так как Granny — не полиморфная. Если мы сделаем `dynamic_cast` к Granny, мы уже не сможем `dynamic_cast`'ом вернуться к Son, потому что `source type is not polymorphic`. При этом `static_cast` мы можем сделать.

17 Специализация шаблонов

Как работает выбор версии между частным и общим, между точной подстановкой и приведением типа? Шаблонные аргументы по умолчанию. Вывод шаблонных параметров и их явное указание при вызове функций. Пример, когда вывести шаблонные параметры не удастся. Как работает и возможна ли перегрузка между `T`, `T&`, `const T&`? Генерируется ли конструктор копирования, если есть конструктор от шаблонного `T&` или `const T&`?

Шаблон — средство языка, предназначенное для кодирования обобщённых алгоритмов и структур без привязки.

Неформально, шаблон — это не готовый код, а рецепт для компилятора, как создавать код в зависимости от переданного типа. Компилятор на ранней стадии компиляции подставляет конкретные типы в шаблон и на выходе получают **специализации**. Данный процесс подстановки называется **инстанцированием**.

Примеры:

```
1 template <typename T>
2 class vector {
3     T* arr;
4     size_t cap;
5     size_t sz;
6 };

1 template <typename First, typename Second>
2 struct pair {
3     First first;
4     Second second;
5 };
```

Замечание. В последнем примере типы `pair<int, int>` и `pair<int, double>` — никак не связанные друг с другом разные типы.

Шаблонные функции и перегрузка

При инстанцировании компилятор выбирает версию функции согласно правилам «**частное лучше общего**» и «**точное соответствие лучше приведения типов**».

Пример. При вызове `f(x)` в `main()` у компилятора на выбор есть две функции. Однако тот выберет первую версию, так как по правилу «точное совпадение лучше приведения типов», в отличие от первой, для второй функции придётся сделать каст к `const int&`.

```
1 template <typename T>
2 void f(T&) {}
3
4 template <typename T>
5 void f(const T&) {}
6
7 int main() {
8     int x = 0;
9     f(x);
10 }
```

Шаблонные аргументы по умолчанию

Шаблоны классов и функций могут иметь аргументы по умолчанию. Если у шаблона есть аргумент по умолчанию, его можно не указать при использовании. Например, шаблон `std::vector` имеет аргумент по умолчанию для аллокатора:

```
1 template <class T, class Allocator = allocator<T>>
2 class vector;
```

При наличии нескольких аргументов шаблона все аргументы после первого аргумента по умолчанию должны иметь аргументы по умолчанию.

Вывод шаблонных параметров

Рассмотрим следующую функцию:

```
1 template <typename T>
2 T Max(const T& x, const T& y) {
3     if (x > y) {
4         return x;
5     }
6     return y;
7 }
```

Ею можно воспользоваться, явно указав параметры шаблона:

```
1 Max <double>(3.14159, 2.71828); //3.14159
2 Max <int>(3.14159, 2.71828); //int-version is called, return 3
```

Однако параметры шаблона в угловых скобках можно не писать, компилятор попытается сам угадать эти параметры по типу аргументов:

```
1 int main() {
2     Max(1, 2); //2, Max<int> is called
3     Max(3.14159, 2.71828); // 3.14159, Max<double> is called
4
5     std::string word1 = "hello", word2 = "world";
6     Max(word1, word2); // world, Max<std::string> is called
7 }
```

Пример, когда вывод шаблонных параметров невозможен

Рассмотрим предыдущую функцию. В случае неоднозначностей, например, при вызове `Max(3.14159, 2)`, компилятор не сможет автоматически вывести параметр, вылетит СЕ. В таком случае, ему придётся указать тип явно: `Max<double>(3.14159, 2)`.

Ещё один пример, когда компилятор не может вывести тип шаблонного параметра:

```
1 template <typename T>
2 void f() {
3     T x;
4     std::cout << x;
5 }
6
7 int main() {
8     f(); //CE
9 }
```

Компилятору неоткуда брать информацию о типе `T`, поэтому будет ошибка компиляции.

Перегрузка между T, T& и const T&

Рассмотрим следующий код:

```
1 template <typename T>
2 void f(T) {
3     std::cout << 1;
4 }
5
6 template <typename T>
7 void f(T&) {
8     std::cout << 2;
9 }
10
11 int main() {
12     int x = 0;
13     f(x); //CE
14     f(0); //OK
15 }
```

Вызов `f(x)` приведёт к ошибке компиляции, поскольку разница между `int` и `int&` в данном случае отсутствует. Однако вызов `f(0)` завершится успешно: `0` — это r-value, а оно никак не может передаваться по ссылке (то есть быть `T&`), выведется 1. Таким образом, полноценная перегрузка между `T` и `T&` невозможна.

Далее, пример с `const T&`:

```
1 template <typename T>
2 void f(T&) {
3     std::cout << 1;
4 }
5
6 template <typename T>
7 void f(const T&) {
8     std::cout << 2;
9 }
10
11 int main() {
12     int x = 0;
13     f(x); //OK
14 }
```

Здесь всё корректно, так как по правилу «полное совпадение лучше приведения типов» для instantiation второй функции компилятору придётся совершить неявный каст к `const T&`, а в случае с первой функции такого не потребуется. Таким образом, вывод будет 1. Перегрузка между `T&` и `const T&` возможна.

Генерация конструктора копирования при наличии конструктора от T& или const T&

Положим, у нас есть класс, в котором мы хотим определить шаблонный конструктор от `T&`. Его определение приведёт к тому, что конструктор копирования класса перестанет вызываться, поскольку шаблонный конструктор от `T&` будет выигрывать у конструктора копирования, который принимает `const T&` по всё тому же правилу «полное совпадение лучше приведения типов». Решением проблемы является создание дополнительного конструктора копирования от просто `T&`.

18 Специализации шаблонов

Синтаксис специализации шаблонов функций. Разница между специализацией и перегрузкой для шаблонных функций. Пример, иллюстрирующий эту разницу. Частичная и полная специализация шаблонов классов. Пример неоднозначности при выборе частичной специализации. Понятие инстанцирования шаблонов, двухэтапная компиляция.

Специализация — конкретная версия шаблона, в которую мы подставляем конкретный тип вместо шаблонного T . Это позволяет нам изменить поведение класса для определённых T .

Полной специализацией называется подстановка конкретных типов вместо всех шаблонных параметров. В таком случае, после слова `template` ставятся пустые угловые скобки (больше нет шаблонных параметров): `template <>`.

Пример. Полная специализация шаблонного класса на примере `vector<bool>`:

```
1 template <typename T>
2 class vector {
3     T* arr;
4     size_t sz;
5     size_t cap;
6 };
7
8 template <> //remove here the substituted parameter T
9 class vector<bool> { //substitute specialised type
10     char* arr;
11     size_t sz;
12     size_t cap;
13 };
```

Мотивация: нам хочется хранить каждый `bool` не в одном байте, а в одном бите, поэтому сделаем (полную) специализацию класса `vector` для $T = \text{bool}$, в котором иначе определим `arr`.

Частичной специализацией называется подстановка конкретных типов вместо некоторого подмножества шаблонных параметров. Несмотря на то, что параметр T может специализироваться, его указание после слова `template` всё ещё может понадобиться.

Пример. Следующий код завершится ошибкой компиляции, поскольку нельзя создавать массив из ссылок (проблема с «висячими ссылками» — «dangling references»):

```
1 template <typename T>
2 struct S {
3     T arr[10];
4 };
5
6 int main() {
7     S<int&> s; //CE
8 }
```

Решением данной проблемы является частичная специализация, которая позволит при создании массива из ссылок хранить в нём не ссылки, а непосредственно сами значения (по правилу «частное лучше общего»):

```

1 template <typename T>
2 struct S {
3     T arr[10];
4 };
5
6 template <typename T>
7 struct S<T&> { //indicating that this class is a (partial) specialisation
8     T arr[10]; //removed the reference using specialisation, create an array
9     by T values
10 };
11
12 int main() {
13     S<int&> s; //OK: the specialisation is called, not the original template
14     class
15 }
```

Пример. Очередной пример частичной специализации класса:

```

1 template <typename T, typename U>
2 struct pair {
3     T x;
4     U y;
5 };
6
7 template <typename T>
8 struct pair<T, T> { //using <...> to indicate that this is a specialisation
9     T x;
10 };
11
```

Замечание. Специализацию можно указывать лишь после определения исходного шаблонного класса, то есть их нельзя менять местами.

Пример неоднозначности при выборе частичной специализации

В следующем коде компилятору не удаётся сделать выбор между первой и второй специализациями, вследствие чего возникает ошибка компиляции:

```

1 template <typename T, typename U>
2 struct pair { //general template function
3     T x;
4     U y;
5 };
6
7 template <typename T>
8 struct pair<T, int> { //first specialisation
9     T x;
10 };
11
12 template <typename T>
13 struct pair<int, T> { //second specialisation
14     T x;
15 };
16
17 int main() {
18     pair<int, int> p; // CE: ambiguous template instantiation
19 }
```

Специализация шаблонных функций: разница между перегрузкой и полной специализацией

(Полной) специализацией функций называется шаблонная функция, начинающаяся со строчки `template <>`, в которой вместо всех шаблонных параметров подставлены конкретные типы.

Замечание. Для функций не бывает частичной специализации: их можно специализировать лишь полностью, а всё остальное будет считаться перегрузкой.

Пример. Рассмотрим следующий пример:

```
1 template <typename T>
2 void f(T&) { //template function
3     std::cout << 1;
4 }
5
6 template <>
7 void f(int&) { //function specialisation
8     std::cout << 2;
9 }
10
11 void f(int) { //function overloading
12     std::cout << 3;
13 }
14
15 int main() {
16     int x = 0;
17     f(x); // output: 3
18 }
```

Здесь компилятор сперва будет выбирать между первой и третьей функциями и выберет нешаблонную. Это значит, что компилятор будет выбирать специализацию лишь после того, как определится с тем, что выбрать следует шаблонную версию функции.

Пример. Аналогичный пример:

```
1 template <typename T, typename U>
2 void f(T, U) { //template function
3     std::cout << 1;
4 }
5
6 template <>
7 void f(int, int) { //specialisation of the previous function
8     std::cout << 2;
9 }
10
11 template <typename T>
12 void f(T, T) { //overloading of the first template function (not partial
13     specialisation!)
14     std::cout << 3;
15 }
16
17 int main() {
18     int x = 0;
19     f(x); //output: 3, because of the rule of specific version
20 }
```

Если же поменять местами вторую и третью функции, то выведется «2», поскольку вновь «частное лучше общего»: сперва компилятор выберет шаблонную функцию «3», а только затем будет решать, какую из её специализаций стоит выбрать.

Понятие инстанцирования шаблонов, двухэтапная компиляция

Инстанцирование — процесс подстановки компилятором конкретных типов вместо шаблонных. Компиляция программ с шаблонными классами или функциями проходит в два этапа:

1. Компилятор просматривает код, в котором ещё не сделаны шаблонные постановки, и проверяет на предмет синтаксических ошибок и таких, которые не зависят от T;
2. Второй раз компилятор просматривает тот же код, но уже инстанцированный (с подстановками конкретных типов вместо T) и, возможно, находит новые ошибки.

Пример. На первом этапе компилятор может обнаружить, например, не поставленную ; в конце строки. На втором же этапе компилятором может быть обнаружена, например, ошибка, связанная с неоднозначностью при выборе частичной специализации, поскольку на втором этапе компилятор уже подставляет конкретные типы вместо шаблонных, что делает возможным обнаружение ошибок такого рода.

Замечание. Замечание. Компилятор делает шаблонные подстановки лениво и по мере надобности:

1. определения функций как правило не инстанцируются, если компилятору достаточно инстанцировать лишь объявление;
2. если инстанцируется шаблонный класс, то тела его методов тоже инстанцируются лишь по мере надобности;
3. если в шаблонном классе есть шаблонный метод, то подстановка шаблонных параметров в метод вообще может не произойти, если к методу не обращались.

19 Простейшие compile-time вычисления

Пример: вычисление чисел Фибоначчи на этапе компиляции. Пример превышения глубины шаблонной рекурсии, параметр `-ftemplate-depth`. Проверка числа на простоту в compile-time с помощью шаблонов.

Пример: вычисление чисел Фибоначчи на этапе компиляции

Compile-time вычислением называется код, способный выполниться ещё на этапе компиляции программы.

Шаблоны в C++ позволяют перенести вычисление некоторых функций прямо во время компиляции благодаря рекурсивному инстанцированию шаблонов.

Пример. Рассмотрим следующий код, вычисляющий n-е число Фибоначчи:

```

1 template <int N>
2 struct Fibonacci {
3     static constexpr int value = Fibonacci<N - 1>::value + Fibonacci<N -
4     2>::value;
5 }
6
7 template <>
8 struct Fibonacci<1> {
9     static constexpr int value = 1;
10 }
11
12 template <>
13 struct Fibonacci<0> {
14     static constexpr int value = 0;
15 }
16
17 int main() {
18     std::cout << Fibonacci<15>::value; // OK: 610
19 }
```

Ключевое слово `constexpr` указывает компилятору, что данную переменную следует вычислить на этапе компиляции. `static` делает переменную `value` общей для всех экземпляров структуры `Fibonacci` на всех этапах рекурсии (хранит её в отдельной области памяти).

Замечание. Код будет работать корректно, если заменить `constexpr` на просто `const`.

Пример превышения глубины шаблонной рекурсии, параметр - `ftemplate-depth`

Если не указать базу рекурсии (две специализации в коде выше), то это приведёт к fatal SE: `template instantiation depth exceeds maximum of 900`, т. е. глубина шаблонной рекурсии будет превышена.

Данное значение максимальной глубины можно вручную увеличить, воспользовавшись параметром компиляции `-ftemplate-depth`. Если же исполнить код выше так же без базы рекурсии, но уже с параметром компиляции `-ftemplate-depth=1000000000`, то это приведёт к тому, что операционная система убьёт компилятор, а сам компилятор получит сигнал `SegFault`:

```

1 g++: internal compiler error: Segmentation fault signal terminated program
  cc1plus
```

Проверка числа на простоту в compile-time с помощью шаблонов

Код следующий:

```
1 template <int N, int D>
2 struct IsPrimeRec { // recursive transition: if not divisible by D and by
   all previous ones
3     static constexpr bool value = (N % D != 0) && IsPrimeRec<N, D - 1>::
   value;
4 };
5
6 template <int N>
7 struct IsPrimeRec<N, 1> { // specialisation for recursion base:
8     // is not divisible by numbers greater than 1, so
   it is prime
9     static constexpr bool value = true;
10 };
11
12 template <int N>
13 struct IsPrimeRec<N, 0> { //specialisation for special case IsPrime<1>
14     static constexpr bool value = false;
15 };
16
17 template <int N>
18 struct IsPrime { //template class for recursion initialisation
19     static constexpr bool value = IsPrimeRec<N, N - 1>::value;
20 };
21
22 int main() {
23     IsPrime<1>::value; //false
24     IsPrime<5>::value; //true
25     IsPrime<10>::value; //false
26 }
```

20 Простейшие метафункции

Метафункция `std::is_same` (проверка равенства двух типов) и ее реализация. Реализация метафункций `add/remove` `const/pointer/reference`. Принципиальная разница между `is_same` и сравнением `typeid`, между `const_cast` и `remove_const`. Алиасы с суффиксами `_v` и `_t` для метафункций. Зависимые имена. Пример неоднозначности между `declaration`'ом и `expression`'ом. Применение ключевого слова `typename` для устранения неоднозначности с зависимым именем.

Рассмотрим следующий код

Рассмотрим следующий код:

```
1 template <typename T>
2 struct S {
3     using A = int;
4 }
5
6 template <>
7 struct S<double> {
8     static const int A = 5;
9 }
10
11 int x = 0;
12
13 template <typename T>
14 void f() {
15     S<T>::A * x; //declaration or expression?!
16 }
```

В данном коде присутствует фундаментальная проблема: компилятор ещё первом этапе (синтаксического парсинга) попытается распознать, что написано в 15-й строчке. Однако в зависимости от выбора специализации, что происходит на втором этапе, эта строчка может быть интерпретирована по-разному: либо как объявление указателя на `int` (тип), либо как выражение `5 * x` (значение). В данном коде выражение `S<T>::A` называется *зависимым именем* (англ. *depending name*).

В стандарте принято соглашение, что выражения такого вида воспринимаются компилятором по умолчанию как значение. Для указания, что это имеено тип, следует написать ключевое слово `typename` перед зависимым именем.

Метафункция `std::is_same`

Метафункция — это языковая конструкция (функция), которая по данному типу возвращает некоторое значение.

`if constexpr (...)` — *определённая языковая конструкция*, позволяющая вычислять условие `if` прямо в `compile-time`, и, кроме того, если это условие ложно, то проверка на корректность тела условного оператора проводится не будет.

Метафункция `std::is_same` проверяет, одинаковые ли типы ей переданы. Вариант её реализации:

```

1 template <typename T, typename U>
2 struct is_same { //specialisation, if different types
3     static constexpr bool value = false;
4 }
5
6 template <typename T>
7 struct is_same<T, T> { //specialisation, if the types are the same
8     static constexpr bool value = true;
9 }
10
11 template <typename T>
12 void f(T x) {
13     if constexpr (is_same<T, int>()) { ... } //compile-time check
14 }
```

Разница между `std::is_same` и сравнением `typeid`

Выражение `typeid(T) == typeid(int)` равносильна метафункции `std::is_same<T, int>`, однако в первом случае выражение будет вычислено в `run-time`, в то время как метафункция узнает значение уже в `compile-time`. Кроме того, если выражение `std::is_same<T, int>` вернёт `false`, то в результирующем машинном коде такого куска кода вообще не будет, поскольку он удалится ещё на этапе компиляции.

Аналогично и с `const_cast` и метафункцией `std::remove_const`.

Реализация метафункций `add/remove const/pointer/reference`

Пример. Реализация `std::remove_reference` (`std::remove_const` и `std::remove_pointer` аналогично):

```

1 template <typename T>
2 struct remove_reference { //if the type is not a reference
3     using type = T;
4 }
5
6 template <typename T>
7 struct remove_reference<T&> { //if the type is a reference
8     using type = T;
9 }
10
11 template <typename T>
12 void g(T x) {
13     typename remove_reference<T>::type y = x; //dependent name!
14 }
```

Замечание. В строчке 13 необходимо указать ключевое слово `typename`, поскольку выражение после него является зависимым именем.

Замечание. В теле метафункции используется конструкция `using A = B`, так как нам необходимо достать сам тип, а не какое-либо произвольное значение.

Пример. Реализация `std::add_const` (`std::add_pointer` и `std::add_reference` аналогично):

```
1 template <typename T>
2 struct add_const {
3     using type = const T;
4 }
5
6 template <typename T>
7 struct add_const<const T> {
8     using type = T;
9 }
```

Алиасы с суффиксами `_v` и `_t` для метафункций

Мотивация: каждый раз при вызове метафункции не удобно писать ключевое слово `typename`. Поэтому в заголовочном файле `type_traits` реализованы алиасы (сокращения) для более лаконичного использования метафункций.

Таким образом, существуют два основных алиаса:

- `_t` (от “type”) возвращает тип, который вернула метафункция;
- `_v` (от “value”) возвращает значение, которое вернула метафункция.

Пример.

```
1 template <typename T>
2 using remove_reference_t = typename remove_reference<T>::type;
3
4 template <typename T>
5 void g(T x) {
6     remove_reference_t<T> y = x;
7 }
8
9 template <typename T, typename U>
10 constexpr bool is_save_v = is_same<T, U>::value;
11
12 template <typename T>
13 void f(T x) {
14     if constexpr (is_same_v<T, int>) { ... }
15 }
```

Замечание. Начиная с C++20 можно не писать `typename` в случаях, как на второй строчке.

21 Шаблоны с переменным количеством аргументов (variadic templates)

Синтаксис использования вариативных шаблонов, примеры. Пакеты аргументов и пакеты параметров, их распаковка. “Откусывание” аргументов по одному. Оператор “sizeof...”.

С C++11 у нас есть возможность пользоваться variadic templates. Это возможность передавать не фиксированное количество аргументов. Синтаксис:

```
1  template<typename... Types> // 1
2  void f(Types... tx) { // 2
3      g(tx...); // 3
4  }
```

Здесь Types — пакет типов. Основное, что мы можем делать с этой пачкой — распаковывать её. Чтобы эту пачку распаковать, нам надо написать многоточие: Types... . Заметим, что многоточие в строке 1 и 2 имеет разный смысл: в строке 1 мы указываем, что мы имеем объявление «пачки» типов, а во 2 строке её распаковка.

Мы можем передать куда-то дальше переданные нам аргументы (строка 3). Чаще всего нам нужно использовать шаблонную рекурсию для работы с шаблонами с переменным количеством параметров. Мы можем «откусить голову» нашему шаблонному параметру. Приведем пример функции, которая выводит переданные ей параметры:

```
1  template <typename Head, typename... Tail>
2  void print(const Head& head, const Tail&... tail) {
3      std::cout << head << '\n';
4      print(tail...);
5  }
```

Заметим две важные вещи: для каждого откусывания головы будет создана отдельная версия функции, так как у нас уменьшается количество переданных аргументов. Такая реализация приведет к СЕ, так как у нас нет выхода из рекурсии и когда у нас tail станет пустым, мы не сможем вызваться. Допишем выход из этой рекурсии:

```
1  void print() {}
2
3  template <typename Head, typename... Tail>
4  void print(const Head& head, const Tail&... tail) {
5      std::cout << head << '\n';
6      print(tail...);
7  }
```

Посмотрим еще раз на наш код: мы пишем const Tail&..., так как мы хотим, чтобы все параметры переданные в Tail были под ссылкой и только потом распаковываем.

Еще один пример использования variadic templates. Реализуем метафункцию is_homogeneous, которая проверяет одинаковые ли типы в переданном нам пакете:

```
1  template <typename First, typename Second, typename... Types>
2  struct is_homogeneous {
3      static constexpr bool value = std::is_same_v<First, Second> &&
is_homogeneous<Second, Types...>::value;
4  };
5
6  template<typename First, typename Second>
7  struct is_homogeneous<First, Second> {
8      static constexpr bool value = std::is_same_v<First, Second>;
9  };
```

В таком случае, у нас && все еще ленивый, вычислять правую часть он не будет, если левая часть ложна, но инстанциироваться данные шаблоны в compile time все равно будут.

Мы можем узнать размер переданного пакета (в «штуках») при помощи оператора sizeof... . То есть если нам был передан pack из 4 переменных, то sizeof... вернет нам 4. Пример:

```
1  void print() {}
2
3  template <typename Head, typename... Tail>
4  void print(const Head& head, const Tail&... tail) {
5      std::cout << sizeof(tail...) << ' ' << sizeof...(Tail...) << ' ';
6      print(tail...);
7  }
8
9  int main() {
10     print(1, 2.f, "biba"); // out: 2 2 1 1 0 0
11 }
```

Спойлер на ~~отныне~~ будущее: с variadic templates есть более удобный способ работы, доступный с C++17: fold expressions.

22 Исключения

Что в реальности происходит при генерации исключения? Двойное копирование при перехвате исключений по значению. Разница между `throw;` и `throw ex;` внутри блока `catch`. Правила выбора блока `catch` для исполнения, разница между ними и перегрузкой функций. Перехват исключений производных классов по ссылке на базовый класс. В какой памяти хранятся объекты исключений?

Что в реальности происходит при генерации исключения? Рассмотрим некоторую структуру `A` и посмотрим на то, что происходит в реальности при бросании исключений:

```
1 struct A {
2     A() { std::cerr << "created A\n" << std::endl; }
3     A(const A&) { std::cerr << "copied A\n" << std::endl; }
4     ~A() { std::cerr << "destroyed A\n" << std::endl; }
5 };
```

Теперь напишем функцию `f`:

```
1 void f() {
2     A a;
3     throw a;
4 }
```

И посмотрим вывод `main`'а:

```
1 int main() {
2     try {
3         f();
4     } catch (A& a) {
5         std::cerr << "caught A" << std::endl;
6     } catch (...) {
7         std::cerr << "caught other" << std::endl;
8     }
9 }
```

```
created A
copied A
destroyed A
caught A
destroyed A
```

Почему вывод именно такой? Понятно, что `A` — скопировалось, поскольку у нас сейчас будет происходить сворачивание стека на текущем уровне, а значит объект будет уничтожен. Далее, уничтожилось `a` в области видимости `f`, было поймано скопированное `a`, и уничтожено после этого. Заметим, что скопированное исключение `a` не могло лежать на стеке, поскольку тогда оно уничтожилось бы при его разворачивании, а значит исключения лежат где-то ещё.

Двойное копирование при перехвате исключений по значению. А теперь посмотрим на вывод программы при перехвате исключения по значению:

```

1 int main() {
2     try {
3         f();
4     } catch (A a) {
5         std::cerr << "caught A" << std::endl;
6     } catch (...) {
7         std::cerr << "caught other" << std::endl;
8     }
9 }

```

```

created A
copied A
destroyed A
copied A
caught A
destroyed A
destroyed A

```

Мы видим, что *a* скопировалось целых два раза: и в момент когда оно бросалось, и в момент когда оно ловилось.

В какой памяти хранятся объекты исключений? Понятно, что исключения не могут лежать на стеке. Но где они тогда могут лежать? Или в статической, или в динамической памяти. Оказывается, в момент создания исключения неявно вызывается `new`/`malloc` и исключение хранится в динамической памяти. Если мы добавим в *f* и ловлю *a* вывод адреса *a*, то мы получим следующее. Видно, что адреса лежат в разных областях памяти.

```

created A
0x7ffd1f87ba30
copied A
destroyed A
0x60d0000000c0
caught A
destroyed A

```

(При этом, *a* мы ловим по ссылке, потому что если ловить по значению, то у нас оно заново создается уже на стеке, и адреса будут мало отличаться). Но что случается в случае если оператор `new` не может выделить память? Он кидает `std::bad_alloc`. Где хранится это исключение? Оно не может храниться в динамической памяти, не может храниться на стеке, а следовательно оно хранится в статической памяти, в заранее зарезервированном компилятором куске памяти.

```

1 int main() {
2     try {
3         try {
4             f();
5         } catch (A& a) {
6             std::cout << "caught A" << std::endl;
7             throw;
8         } catch (...) {
9             std::cout << "caught other" << std::endl;
10        }
11    } catch (A& a) {
12        std::cout << "caught A again" << std::endl;

```

```

13     }
14 }

```

При запуске такого кода мы получим следующий результат: Что он означает? Уже со-

```

created A
copied A
destroyed A
caught A
caught A again
destroyed A

```

зданное исключение *a* пробросится дальше, а у ещё раз пойманного *A* будет тот же адрес, что и у первого. Но если бы мы там написали вот так:

```

1 int main() {
2     try {
3         try {
4             f();
5         } catch (A& a) {
6             std::cout << "caught A" << std::endl;
7             throw a;
8         } catch (...) {
9             std::cout << "caught other" << std::endl;
10        }
11    } catch (A& a) {
12        std::cout << "caught A again" << std::endl;
13    }
14 }

```

```

created A
copied A
destroyed A
caught A
copied A
destroyed A
caught A again
destroyed A

```

То исключение будет заменено новым исключением, и создастся ещё раз. Т.е. `throw a;` имеет смысл создания нового исключения, а `throw` – пробрасывания старого на уровень выше.

```

1 int main() {
2     try {
3         f();
4     } catch (A& a) {
5         std::cerr << "caught A" << std::endl;
6         throw a ;
7     } catch (...) {
8         std::cerr << "caught other" << std::endl;
9     }
10 }

```

При непойманном исключении у нас возникает *Unspecified Behavior*, которое заключается в том, что компилятор не гарантирует вызов деструкторов объектов.

```
created A
copied A
destroyed A
caught A
copied A
terminate called after throwing an instance of 'A'
```

```
1 void f() {
2     throw 1;
3 }
4
5 int main() {
6     try {
7         f();
8     } catch (double) {
9         std::cerr << "caught double" << std::endl;
10    } catch (unsigned int) {
11        std::cerr << "caught unsigned int" << std::endl;
12    } catch (int) {
13        std::cerr << "caught int" << std::endl;
14    }
15 }
```

```
caught int
```

Заметим, что при выборе блока `catch` у нас выбирается не более одного блока. При этом, если есть в точности совпадающий блок `catch`, то выполнится именно он. Предположим, мы уберем этот блок. Что тогда у нас выведется?

```
1 int main() {
2     try {
3         f();
4     } catch (double) {
5         std::cerr << "caught double" << std::endl;
6     } catch (unsigned int) {
7         std::cerr << "caught unsigned int" << std::endl;
8     }
9 }
```

```
terminate called after throwing an instance of 'int'
```

Из этого примера становится понятно, что `catch` не умеет пользоваться кастами (с оговорочками). Блок выбирается по принципу или точное соответствие, или ничего, за исключением снятия константной ссылки/ссылки/константности с величины. И этим он отличается от перегрузки функций.

Перехват исключений производных классов по ссылке на базовый класс. Но есть и ещё одно исключение из правил: мы можем словить исключение класса-наследника по ссылке на класс-родитель.

```
1 struct Base {
2     Base() { std::cerr << "created Basen" << std::endl; }
3     Base(const Base&) { std::cerr << "copied Base" << std::endl; }
4     ~Base() { std::cerr << "destroyed Base" << std::endl; }
5 };
6
7 struct Derived : Base {
8     Derived() { std::cerr << "created Derived" << std::endl; }
9     Derived(const Derived&) { std::cerr << "copied Derived" << std::endl; }
```

```

10 ~Derived() { std::cerr << "destroyed Derived" << std::endl; }
11 };

```

Теперь посмотрим на результат выполнения следующего кода:

```

1 void f() {
2     Derived d;
3     throw d;
4 }
5
6 int main() {
7     try {
8         f();
9     } catch (Base b) {
10        std::cerr << "catch Base" << std::endl;
11    } catch (Derived d) {
12        std::cerr << "catch Derived" << std::endl;
13    }
14 }

```

```

created Base
created Derived
created Base
copied Derived
destroyed Derived
destroyed Base
copied Base
catch Base
destroyed Base
destroyed Derived
destroyed Base

```

При этом, поскольку у нас ловится первое точное совпадение, то у нас второй catch — бесполезен. Что у нас происходит? В функции *f* у нас создан объект класса Derived, соответственно была создана его база; После, у нас создано исключение, был удалён локальный Derived, пойман Base по значению, удалён, и было удалено исключение, хранящееся в динамической памяти. Если поймать по ссылке — то ничего особо не изменится, просто уменьшится число лишних копирований. Но что будет, если сделать наследование приватным?

```

1 struct Derived : private Base {
2     Derived() { std::cerr << "created Derived" << std::endl; }
3     Derived(const Derived&) { std::cerr << "copied Derived" << std::endl; }
4     ~Derived() { std::cerr << "destroyed Derived" << std::endl; }
5 };

```

```

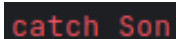
created Base
created Derived
created Base
copied Derived
destroyed Derived
destroyed Base
created Base
copied Derived
catch Derived
destroyed Derived
destroyed Base
destroyed Derived
destroyed Base

```

Тогда у нас поймается именно Derived! То есть, catch не видит private-наследование. Также, есть очень интересный случай с бабушкой и ромбовидным наследованием, а именно, у нас

не поймается бабушка, поскольку у нас каст неоднозначен. Т.е. результатом следующего кода будет:

```
1 struct Granny {};  
2  
3 struct Mom : Granny {};  
4  
5 struct Dad : Granny {};  
6  
7 struct Son : Mom, Dad {};  
8  
9 void f() {  
10     Son s;  
11     throw s;  
12 }  
13  
14 int main() {  
15     try {  
16         f();  
17     } catch (Granny g) {  
18         std::cerr << "catch Granny" << std::endl;  
19     } catch (Son s) {  
20         std::cerr << "catch Son" << std::endl;  
21     }  
22 }
```



При этом, виртуальное наследование всё чинит, поскольку делает каст однозначным.

23 Исключения в конструкторах и деструкторах

Особенности исключений в конструкторах. Исключения в списках инициализации конструкторов, function-try блоки и их применение. В чем главная проблема исключений в деструкторах? Применение функции `std::uncaught_exception`.

Особенности исключений в конструкторах. Пусть у нас есть вот такие структуры:

```

1 struct A {
2     int x;
3     A(int x): x(x) { std::cerr << "created A " << x << std::endl; }
4     A(const A&) { std::cerr << "copied A" << std::endl; }
5     ~A() { std::cerr << "destroyed A " << x << std::endl; }
6 };
7
8 struct S {
9     A a;
10    S(int x): a(x) {
11        std::cerr << "created S" << std::endl;
12        if (x == 0) {
13            throw 1;
14        }
15    }
16    ~S() {
17        std::cerr << "destroyed S" << std::endl;
18    }
19 };

```

Тогда, запустим на ней вот такой код:

```

1 int main() {
2     try {
3         S(0);
4     } catch (...) {
5         std::cerr << "caught" << std::endl;
6     }
7 }

```

```

created A 0
created S
destroyed A 0
caught

```

При возникновении исключения в конструкторе не очень понятно, как будут выглядеть вызову деструкторов объектов. Деструктор *S* вызван не будет, но при этом компилятор должен обеспечить корректное уничтожение всего лежащего и уже созданного внутри, т.е. деструктор объекта класса *A* будет вызван.

```

1 struct S {
2     A a;
3     int* p;
4     S(int x): a(x), p(new int(0)) {
5         std::cerr << "created S" << std::endl;
6         if (x == 0) {
7             throw 1;
8         }
9     }
10    ~S() {

```

```

11     delete p;
12     std::cerr << "destroyed S" << std::endl;
13 }
14 };

```

Поэтому такой код провоцирует утечку памяти, поскольку объект S — недосоздан, и следовательно его деструктор не будет вызван.

Исключения в списках инициализации конструкторов Но сразу возникает вопрос: а что происходит, если исключение бросается в списке инициализации?

```

1 struct A {
2     int x;
3     A(int x): x(x) {
4         std::cerr << "created A " << x << std::endl;
5         if (x == 0) {
6             throw 1;
7         }
8     }
9     A(const A&) { std::cerr << "copied A" << std::endl; }
10    ~A() { std::cerr << "destroyed A " << x << std::endl; }
11 };
12
13 struct S {
14     A a0;
15     A a1;
16     A a2;
17     S(int x): a0(x + 1), a1(x), a2(x + 2) {
18         std::cerr << "created S" << std::endl;
19         if (x == 0) {
20             throw 1;
21         }
22     }
23     ~S() {
24         std::cerr << "destroyed S" << std::endl;
25     }
26 };

```

Мы видим, что у нас опять вызывается деструкторы объектов, созданных полностью, то есть $a(1)$, а до последнего A и, уж тем более, до тела конструктора S дело не доходит; деструктор объекта, в конструкторе которого было брошено исключение тоже не вызовется.

```

created A 1
created A 0
destroyed A 1
caught

```

function-try блоки и их применение Но писать постоянно где-то снаружи `try...catch` блоки — так себе идея, противоречащая мотивировке исключений, и, + `main` не обязан знать про внутреннее устройство структуры. Поэтому, для обработки исключений в функциях существует специальный синтаксис, называемый `function-try` блоками:

```

1 // function-try block
2 void f() try {
3     // some code
4 } catch (...) {
5     // some code
6 }

```

Тогда, перепишем конструктор S с учётом этой возможности:

```

1 struct S {
2     A a0;

```

```

3   A a1;
4   A a2;
5   S(int x) try: a0(x + 1), a1(x), a2(x + 2) {
6       std::cerr << "created S" << std::endl;
7       if (x == 0) {
8           throw 1;
9       }
10  } catch (...) { std::cerr << "caught in S!" << std::endl; }
11  ~S() {
12      std::cerr << "destroyed S" << std::endl;
13  }
14 };

```

```

created A 1
created A 0
destroyed A 1
caught in S!
caught

```

При этом, если не писать `throw`;/`throw ex`; в конце `function-try` блока в списке инициализации, он в любом случае будет неявно написан и проброшен, поскольку его отсутствие — не имеет смысла, поскольку у нас объект, в списке инициализации которого произошла ошибка недосоздан.

В чем главная проблема исключений в деструкторах? Начиная с C++11 принято, что декструкторы исключения не кидают (в подавляющем большинстве случаев).

```

1 struct B {
2     ~B() {
3         throw 1;
4     }
5 };
6
7 int main() {
8     try {
9         B;
10    } catch (...) {
11        std::cerr << "caught" << std::endl;
12    }
13 }

```

```

plusplus/tex/exam.cpp:6:5: warning: 'throw' will always call 'terminate' [-Wterminate]
   6 |     throw 1;
     |     ~~~~~~
plusplus/tex/exam.cpp:6:5: note: in C++11 destructors default to 'noexcept'

```

И, как следствие, вызовется: Почему это так? Деструктор обладает одним важным свой-

```
terminate called after throwing an instance of 'int'
```

ством: он может вызваться не только по причине штатного завершения работы, но и по причине того, что кинули исключение. (При этом кинуть и поймать исключение в деструкторе можно). А можно написать вот так:

```

1 struct B {
2     B()=default;
3     ~B() noexcept(false) {
4         throw 1;

```

```
5 }
6 };
```

И теперь кидать exception'ы можно, но, при этом, если наше исключение будет вторым исключением, т.е. мы попали в деструктор из-за исключения, то мы всё равно получим terminate.

Применение функции `std::uncaught_exception`. Что делать, если мы вдруг захотели в данных текущий момент времени узнать о том, выполняется ли наша программа штатно? Для этого у нас есть функция `std::uncaught_exception` — реализованная на уровне компилятора, самим её сделать нельзя. Тогда можно сделать вот так:

```
1 struct B {
2     B()=default;
3     ~B() noexcept(false) {
4         if (!std::uncaught_exceptions()) {
5             throw 1;
6         }
7     }
8 };
```

Тогда вот на следующем код у нас будет реакция «caught»:

```
1 int main() {
2     try {
3         B b;
4         throw 1;
5     } catch (...) {
6         std::cerr << "caught" << std::endl;
7     }
8 }
```

А вот такой код вызывает terminate:

```
1 struct B {
2     B()=default;
3     ~B() noexcept(false) {
4         /*     if (!std::uncaught_exceptions()) {
5             throw 1;
6         }*/
7         throw 1;
8     }
9 };
10
11 int main() {
12     try {
13         B b;
14         throw 1;
15     } catch (...) {
16         std::cerr << "caught" << std::endl;
17     }
18 }
```

Заметим, при этом, что функция `std::uncaught_exception` — deprecated с C++17, removed в C++20, а её заменила функция `std::uncaught_exceptions` — возвращающая количество ещё не пойманных исключений в текущем потоке. Функция, возвращающая bool была заменена на возвращающую int, поскольку у нас может быть одновременно в runtime сразу несколько исключений.

24 Безопасность относительно исключений, спецификации исключений

Спецификатор `noexcept`. Условный `noexcept`. Оператор `noexcept`, его действие и использование в комбинации со спецификатором `noexcept`. Понятие `exception-safety`, понятия `strong` и `basic exception guarantee`.

Понятие `exception-safety`, понятия `strong` и `basic exception guarantee`. Когда мы пишем код на C++ — мы стоим на минном поле, где в любой момент какая-то фигня может сделать нам плохо; например, если мы пишем шаблонный класс над T , то нам теперь, после изучения исключений, стоит учитывать, что T тоже может нам кинуть исключение, и любая функция может выкинуться ранее `return`'а. Код, написанный так, чтобы работать в таких условиях, называется безопасным относительно исключений.

Существует несколько категорий безопасности функции относительно исключений:

- ▷ `Nothrow (nofail)` — функция не кидает исключений
- ▷ `Strong Exception Guarantee` — функция гарантирует, что в случае прокидывания ошибки у нас всё вернётся в состояние непосредственно до начала вызова этой функции
- ▷ `Basic Exception Guarantee` — если будет прокинуто исключение, то программа будет находиться в валидном состоянии, никакие ресурсы не утекут и будут сохранены все инварианты программы
- ▷ `No Exception Guarantee` — произойти с программой может что угодно.

Кроме того, есть ещё `exception-neutral guarantee`; её предоставляют некоторые стандартные функции. Она заключается в том, что если было вызвано исключение шаблонным параметром, то оно в неизменном виде пробросится вверх (например, `Compare` в `std::sort` и конструктор класса T в `std::make_shared`).

Спецификатор `noexcept`. Мы можем пометить функцию ключевым словом `noexcept`, которое скажет компилятору, что эта функция не должна бросать исключению из себя, т.е. мы гарантируем, что любое исключение, брошенное внутри функции будет поймано в ней же. Если же из помеченной `noexcept` функции будет проброшено исключение будет вызван `std::terminate`.

```
1 void f() noexcept {  
2     // some code  
3 }
```

Спецификатор `noexcept` следует понимать не просто как гарантию отсутствия пробрасывания исключений из функции, но и как более сильное условие: мы гарантируем, что функция не может пойти не так. Поэтому, допустим, стандартный метод `size` контейнеров — `noexcept`, поскольку в нём ничего сломаться не может. А, вот, `operator[]` у, допустим, вектора — не является `noexcept`.

Условный noexcept. Пусть у нас есть шаблонная функция, и мы хотим, чтобы она не бросала исключения зависимости от каких-либо условий. Это возможно реализовать при помощи условного noexcept'a. Например,

```
1 template<typename T>
2 void f() noexcept(std::is_reference_v<T>) {
3     \\ some code
4 }
```

Оператор noexcept, его действие и использование в комбинации со спецификатором noexcept. Часто требуется сделать функцию noexcept в зависимости от того, является ли noexcept вызов другой функции. Для этого существует оператор noexcept, которые в compile-time проверяет является функция noexcept или нет. Если он вызывается для функции, то он проверяет есть ли у этой функции спецификатор noexcept, а если для какого-то стандартного оператора — то все стандартные операторы кроме new, typeid, dynamic_cast и throw являются noexcept.

```
1 template<typename T>
2 void g() {}
3
4 template<typename T>
5 void f() noexcept {}
6
7 int main() {
8     std::cout << noexcept(g<int>());
9     std::cout << noexcept(f<int>());
10 }
```

И такой код выведет 01.

Теперь, для того чтобы сделать функцию f noexcept если и только если g — noexcept, мы должны внутри specifier noexcept прописать вызов operator noexcept:

```
1 template<typename T>
2 void g() {}
3
4 template<typename T>
5 void f() noexcept(noexcept(g<T>())) {}
6
7 int main() {
8     std::cout << noexcept(g<int>());
9     std::cout << noexcept(f<int>());
10 }
```

Т.е. если мы запустим вот такой код:

```
1 template<typename T>
2 void g() noexcept(std::is_same_v<T, int>) {}
3
4 template<typename T>
5 void f() noexcept(noexcept(g<T>())) {}
6
7 int main() {
8     std::cout << noexcept(g<int>());
9     std::cout << noexcept(f<int>());
10    std::cout << noexcept(g<double>());
11    std::cout << noexcept(f<double>());
12 }
```

То у нас выведется 1100.

25 Итераторы

Пример реализации итераторов для класса `vector`. Константные и неконстантные итераторы. Избавление от копирования кода при реализации константных и неконстантных итераторов.

Reverse-итераторы, класс `std::reverse_iterator`, его реализация.

Инвализация итераторов. Класс `std::iterator_traits`, его предназначение и использование. Функции `std::advance` и `std::distance`, различие в поведении этих функций для разных видов итераторов, реализация этого различия.

Класс `std::iterator_traits`, его предназначение и использование. Предположим, нам внезапно захотелось сделать шаблонную функцию с шаблоном, который является итератором, и там нам неожиданно понадобилось разыменовать итератор. Но `auto` нам в данном случае может сделать плохо, потому что при разыменовании итератора по `vector<bool>` мы получим `__Bit_reference` — что не есть гуд. Помочь с этим вопросом может `std::iterator_traits`:

```
1 template<typename InputIterator>
2 void some_iterator_funciton(InputIterator begin, InputIterator end) {
3     typename std::iterator_traits<InputIterator>::value_type x = *begin;
4 }
```

Кроме того, при помощи `iterator_traits` мы можем ещё узнать: `difference_type`, `value_type`, `pointer`, `reference`, `iterator_category`. И вот `iterator_category` — вторая очень полезная часть этого класса. Это тип, который обозначает категорию итератора и является просто тегами, т.е. типами, которые нужны для проверки их на равенство с другими типами.

```
struct input_iterator_tag {}; (1)
struct output_iterator_tag {}; (2)
struct forward_iterator_tag : public input_iterator_tag {}; (3)
struct bidirectional_iterator_tag : public forward_iterator_tag {}; (4)
struct random_access_iterator_tag : public bidirectional_iterator_tag {}; (5)
struct contiguous_iterator_tag : public random_access_iterator_tag {}; (6) (since C++20)
```

Заметим, что у `iterator_traits` есть специализация для T^* . Она нужна, поскольку функции для того, чтобы достать нужные типы по сути просто смотрят в итератор. Но если T — указатель, то у нас уже на этом моменте произойдёт ошибка.

Функции `std::advance` и `std::distance`, различие в поведении этих функций для разных видов итераторов, реализация этого различия. В стандартной библиотеке есть функция `std::distance`, которая считает количество «прыжков» от первого итератора до второго. Попробуем написать её реализацию:

```
1 template<typename Iter>
2 \* return type *\ distance(Iter first, Iter last);
```

Какой должен быть возвращаемый тип у этой функции?

Он должен быть `typename std::iterator_traits<Iter>::difference_type`.

```
1 template<typename Iter>
2 typename std::iterator_traits<Iter>::difference_type distance(Iter first,
    Iter last);
```

Понятно, что поведение итератора должно отличаться для `RandomAccess` итераторов и остальных. Для реализации этого различия воспользуемся `if constexpr`:

```

1 template<typename Iter>
2 typename std::iterator_traits<Iter>::difference_type distance(Iter first,
   Iter last) {
3     if constexpr (std::is_base_of_v<std::random_access_iterator_tag, typename
   std::iterator_traits<Iter>::iterator_category>) {
4         return last - first;
5     }
6     typename std::iterator_traits<Iter>::difference_type result = 0;
7     while (first != last) {
8         ++first;
9         ++result;
10    }
11    return result;
12 }

```

Теперь distance будет работать за константу для RandomAccess и Contiguous, и за линейно для остальных категорий.

Ещё есть функция std::advance, которая сдвигает итератор на n прыжков. И тут тоже понятно, что поведение функции будет различно для различных категорий итераторов:

```

1 template<typename Iter, typename Distance>
2 constexpr void advance(Iter& iter, Distance n) {
3     using category = typename std::iterator_traits<Iter>::iterator_category;
4
5     auto dist = typename std::iterator_traits<Iter>::difference_type(n);
6
7     if constexpr (std::is_base_of_v<std::random_access_iterator_tag, category>) {
8         iter += dist;
9         return;
10    }
11
12    if constexpr (dist < 0 and std::is_same_v<std::bidirectional_iterator_tag,
   category>) {
13        while (dist < 0) {
14            --iter;
15            ++dist;
16        }
17        return;
18    }
19
20    while (dist > 0) {
21        ++iter;
22        --dist;
23    }
24 }

```

Пример реализации итераторов для класса vector. Попробуем реализовать итераторы для vector:

```

1 template<typename T>
2 class vector {
3     T* arr_;
4     std::size_t sz_;
5     std::size_t cap_;
6 public:
7     class iterator {
8     private:
9         T* ptr;
10        iterator(T* ptr): ptr(ptr) {}

```

```

11 public:
12     iterator(const iterator&)=default;
13     iterator& operator=(const iterator&)=default;
14
15     iterator& operator+=(std::ptrdiff_t diff) { /* implementation */}
16     /*
17     other contiguous iterator methods
18     */
19
20     T& operator*() const { return *ptr; }
21
22     T* operator->() const { return ptr; }
23
24     iterator& operator++() {
25         ++ptr;
26         return *this;
27     }
28
29     iterator operator++(int) {
30         iterator copy = *this;
31         ++ptr;
32         return copy;
33     }
34 };
35
36 iterator begin() { return iterator{arr_}; }
37 iterator end() { return iterator{arr_ + sz_}; }
38 };

```

Примерно так на примитивном уровне можно написать итератор для вектора. (Понятно дело, что это неоч реализация)

Константные и неконстантные итераторы. Избавление от копирования кода при реализации константных и неконстантных итераторов. Но как можно сделать const_iterator, т.е. такой итератор, что лежащий под ним элемент менять нельзя? Для этого можно сделать отдельный класс const_iterator, в котором просто хранить const T*, но это кринж. Для того, чтобы сделать это по нормальному, надо в private засунуть шаблонным класс base_iterator с булевским шаблоном и прописать внутри conditional using'и, а в public определить iterator и const_iterator через using'и:

```

1 template<typename T>
2 class vector {
3     T* arr_;
4     std::size_t sz_;
5     std::size_t cap_;
6
7     template<bool IsConst>
8     class base_iterator {
9     public:
10         using difference_type = std::ptrdiff_t;
11         using value_type = T;
12         using pointer = std::conditional_t<IsConst, const T*, T*>;
13         using reference = std::conditional_t<IsConst, const T&, T&>;
14         using iterator_category = std::contiguous_iterator_tag;
15
16     private:
17         pointer ptr_ = nullptr;
18         explicit base_iterator(pointer ptr): ptr_(ptr) {}
19

```

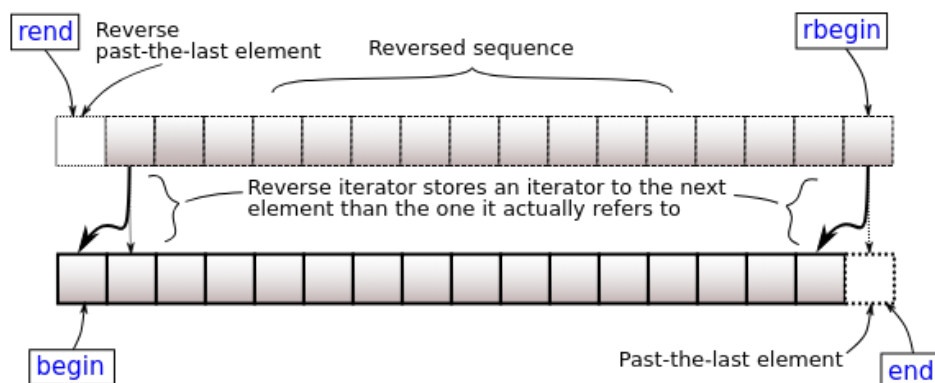
```

20     public:
21         /*
22             iterator methods...
23         */
24     };
25     public:
26     using iterator = base_iterator<false>;
27     using const_iterator = base_iterator<true>;
28 };

```

Ещё один момент который стоит заметить — это `begin/end`, `const` версии который теперь должны возвращать `const_iterator`'ы.

Reverse-итераторы, класс `std::reverse_iterator`, его реализация. Ещё у контейнеров есть тип `reverse_iterator` — адаптер для итератора, который его разворачивает. (естественно, только для контейнеров, категория итераторов у которых хотя бы `Bidirectional`) Есть следующие требования к разыменованию и стрелочке для итератора: А реализация



reference operator*() const;	(1)	(until C++17)
constexpr reference operator*() const;		(since C++17)
pointer operator->() const;		(until C++17)
constexpr pointer operator->() const;		(since C++17)
constexpr pointer operator->() const	(2)	(until C++20)
requires (std::is_pointer_v<Iter>		(since C++20)
requires (const Iter i) { i.operator->(); });		

Returns a reference or pointer to the element previous to current.

1) Equivalent to <code>Iter tmp = current; return *--tmp;</code>	
2) Equivalent to <code>return std::addressof(operator*());</code>	(until C++20)
2) Equivalent to <code>return current - 1;</code> if <code>Iter</code> is a pointer type. Otherwise, equivalent to <code>return std::prev(current).operator->();</code>	(since C++20)

`reverse_iterator` может выглядеть примерно вот так:

```

1  template<typename Iter>
2  class reverse_iterator {
3  public:
4      using iterator_category = std::iterator_traits<Iter>::iterator_category;
5      using value_type = std::iterator_traits<Iter>::value_type;
6      using difference_type = std::iterator_traits<Iter>::difference_type;
7      using pointer = std::iterator_traits<Iter>::pointer;
8      using reference = std::iterator_traits<Iter>::reference;
9
10     template<typename Other>
11     static constexpr bool convertible = !std::is_same_v<Other, Iter> && std::
        convertible_to<const Other&, Iter>;
12     protected:

```

```

13  Iter current = Iter();
14  public:
15  reverse_iterator(): current() {}
16
17  constexpr explicit reverse_iterator(Iter iter): current(iter) {}
18
19  reverse_iterator(const reverse_iterator& other): current(other) {} // copy
    constructor + template copy constructor
20  template<typename Other> requires (convertible<Other>)
21  reverse_iterator(const reverse_iterator<Other>& other): current(other.
    current) {}
22
23  reverse_iterator& operator=(const reverse_iterator&)=default;
24  template<typename Other> requires (convertible<Other> && std::
    assignable_from<Iter&, const Other&>)
25  reverse_iterator& operator=(const reverse_iterator<Other>& other) {
26      current = other.current;
27      return *this;
28  }
29
30  constexpr Iter base() const { return current; }
31
32  constexpr reference operator*() const { Iter tmp = current; return *--tmp;
    } // return previous elem
33
34  pointer operator->() const {
35      return std::prev(current).operator->(); // :(
36  }
37
38  constexpr reverse_iterator& operator++() { // reverse ++
39      --current;
40      return *this;
41  }
42
43  constexpr reverse_iterator operator++(int) {
44      auto copy = *this;
45      ++(*this);
46      return copy;
47  }
48
49  constexpr reverse_iterator& operator--() { // reverse --
50      ++current;
51      return *this;
52  }
53
54  constexpr reverse_iterator operator--(int) {
55      auto copy = *this;
56      --(*this);
57      return copy;
58  }
59
60  // Other methods (for RandomAccess) defining similarly
61 };
62 };

```

Почему такая реализация? Потому что ~~срreference~~ `reference`. Мы храним в адаптере итератор, и при выполнении какой-то операции выполняем над внутренним противоположную; нам надо, чтобы `begin` было последним элементом контейнера, а `end` указывало за первый; т.е. мы должны возвращать при разыменовании предыдущий перед текущим.

Инвализация итераторов. Методы контейнеров могут инвалидировать некоторые итераторы при их использовать (а могут и не инвалидировать). Например, вот такой код является UB:

```
1 int main() {
2     std::vector<int> v(10);
3     std::vector<int>::iterator x = v.begin() + 5;
4     v.push_back(1);
5     std::cout << *x;
6 }
```

Вообщем-то, для контейнеров есть следующая табличка по инвалидации итераторов:

Category	Container	After insertion , are...		After erasure , are...		Conditionally
		iterators valid?	references valid?	iterators valid?	references valid?	
Sequence containers	array	N/A		N/A		
	vector	No		N/A		Insertion changed capacity
		Yes		Yes		Before modified element(s) (for insertion only if capacity didn't change)
	deque	No		No		At or after modified element(s)
		No	Yes	Yes, except erased element(s)		Modified first or last element
			No	No	No	
	list	Yes		Yes, except erased element(s)		
	forward_list	Yes		Yes, except erased element(s)		
Associative containers	set multiset map multimap	Yes		Yes, except erased element(s)		
Unordered associative containers	unordered_set unordered_multiset unordered_map unordered_multimap	No	Yes	N/A		Insertion caused rehash
		Yes		Yes, except erased element(s)		No rehash

26 Внутреннее устройство std::vector

Поля класса vector. Реализация reserve (без поддержки move-семантики).

Реализация push_back.

Разница между [] и методом at().

Класс vector<bool> и его отличие от обычного vector, преимущества и недостатки.

Реализация оператора [] для vector<bool>.

Правила инвалидации итераторов, указателей и ссылок для vector.

Поля класса vector

```
1 template <typename T>
2 class vector {
3     T* arr_;
4     size_t sz_;
5     size_t cap_;
6     // ...
7 }
```

У вектора есть 3 поля:

- ▷ Поле типа T* хранит указатель на начало данных, хранимых в векторе
- ▷ Поле size хранит текущий размер вектора, то есть количество хранимых объектов
- ▷ Поле capacity хранит вместимость вектора = размер буфера, в котором хранятся данные. Другими словами количество объектов, которые поместятся в выделенную память = размер динамически выделенного массива, на который указывает первое поле

Реализация reserve (без поддержки move-семантики)

```
1 void reserve(size_t newcap) {
2     if (newcap <= cap_) {
3         return;
4     }
5     T* newarr = reinterpret_cast<T*>(new char[newcap * sizeof(T)]);
6     size_t oldindex = 0;
7     try {
8         for (; oldindex < sz_; ++oldindex) {
9             new (newarr + oldindex) T(arr_[oldindex]); // placement new: copy-
               construct object at address newarr + oldindex from arr_[oldindex]
10        }
11    } catch (...) {
12        // 1)
13        for (size_t index = 0; index < oldindex; ++index) {
14            (newarr + index)->~T();
15        }
16
17        // 2)
18        delete[] reinterpret_cast<char*>(newarr);
19
20        // 3)
21        throw;
22    }
```

```

22 }
23 for (size_t index = 0; index < sz_; ++index) {
24     (arr_ + index)->~T();
25 }
26 delete[] reinterpret_cast<char*>(arr_);
27 arr_ = newarr;
28 cap_ = newcap;
29 }

```

Пояснения к коду:

- ▷ Сначала мы выделяем новый кусок памяти запрошенного размера. Если выделять память с помощью `new T[newcap]`, то мы, как и хотим, получим массив нужного размера и типа, но будут вызваны default-конструкторы всех объектов в этом массиве. Чтобы этого избежать и запросить «сырую» память, мы пользуемся тем, что тип `char` всегда имеет размер в 1 байт и что никакие конструкторы при выделении массива `char`'ов не вызываются. Итого, мы, с помощью `new` выделяем в куче `newcap * sizeof(T)` байт (а точнее `char`'ов) сырой памяти, то есть там лежит мусор, а потом, с помощью `reinterpret_cast` складываем указатель на начало этой памяти в переменную `newarr`. Теперь там лежит указатель на блок памяти нужного размера, а так как тип у этой переменной `T*`, индексация будет работать правильно.
- ▷ Теперь пришло время «перенести» объекты из старого буфера в новый. Здесь по-хорошему стоит использовать move-семантику, но пока мы обойдёмся без неё, а сделаем это с помощью копирования: просто скопируем все объекты из старого буфера в новый поочередно. Для этого нам понадобится новый синтаксис под названием «placement new». Заметим, что раньше мы всегда вызывали конструктор объекта сразу вместе с выделением памяти под него: мы либо создавали объект в автоматической памяти, либо одновременно выделяли память под объект в куче и конструировали его с помощью обычного `new` (то есть писали `T* ptr = new T(args);`). Сейчас же перед нами стоит задача вызвать конструктор объекта на уже имеющейся сырой памяти. Именно для этого и нужен «placement new». Синтаксис такой:

```
1 new (ptr) Type(args_for_ctor);
```

- ▷ Нам нужно обеспечить strong exception guarantee: если где-нибудь было выброшено исключение, то нужно вернуть всё как было и пробросить его дальше. Вылететь исключение в нашей функции `reserve` может только в двух местах: в 5 строке, где мы выделяем память с помощью `new` и в 9 строке, где мы вызываем сору-конструктор `T`. К 5 строке мы ничего ещё не успели сделать, поэтому если `new` в 5 строке выкинет исключение, то гарантии будут соблюдены: память выделиться не смогла, а значит не утекла, а так как мы не в блоке `try`, то функция завершится и исключение полетит выше, в вызывающую функцию.

Если какой-нибудь из сору-конструкторов кинул исключение, то нужно 1) вызвать деструкторы всех объектов, которые мы успели сконструировать в новом динамическом массиве 2) `delete`'нуть этот новый буфер, то есть освободить выделенную под него память, вернуть эту память системе 3) пробросить выброшенное исключение дальше. К счастью, так как объекты в новом массиве мы конструировали как копии объектов из старого, объекты из старого никак не «повредились», поэтому тут всё ок.

- ▷ Наконец, если все предыдущие шаги успешно пройдены, осталось вызвать деструкторы объектов в старом массиве, освободить занимаемую им память и обновить указатель и `capacity`. Так как память мы выделяли как массив `char`'ов, то при удалении надо сначала обратно скастить наш массив к массиву `char`'ов.

Реализация push_back

```

1 void push_back(const T& value) {
2     if (sz_ == cap_) {
3         reserve(cap_ > 0 ? cap_ * 2 : 1);
4     }
5     new (arr_ + sz_) T(value);
6     ++sz_;
7 }

```

Разница между [] и методом at()

Разница между ними заключается в том, что обращение к некорректному индексу с помощью operator[] — это UB, а если обратиться за границы массива с помощью метода at(), то метод at() кинет исключение типа std::out_of_range

Другими словами, метод at() выполняет проверки, вследствие чего он позволяет упростить обнаружение и исправление ошибок, но работает немного медленнее, чем квадратные скобки.

Класс vector<bool> и его отличие от обычного vector, преимущества и недостатки

Использование общей реализации vector будет сохранять каждый элемент в отдельный объект типа bool, который весит 1 байт, то есть 8 бит. Используя битовые операции, можно в 1 байте сохранить информацию о 8 bool'ах, чем хочется воспользоваться, чтобы не тратить в 8 раз больше памяти. Но проблема в том, что вернуть по квадратным скобкам ссылку на bool не получится, так как тогда мы вернём ссылку на 1 байт, а хотим на 1 бит. Нам понадобится завести тип обёртку, через который можно будет считывать значение отдельного бита и записывать в него. А квадратные скобки будут создавать объект такого типа и возвращать его.

Плюсы такой специализации перед общей: занимаемая память в 8 раз меньше

Минусы такой специализации: квадратные скобки возвращают rvalue

Реализация оператора [] для vector<bool>

```

1 template <>
2 class Vector<bool> {
3     char* arr_;
4     size_t sz_;
5     size_t cap_;
6
7     struct BitReference {
8         char* cell;
9         uint8_t index;
10
11         BitReference(char* cell, uint8_t index) : cell(cell), index(index) {}
12
13         BitReference& operator=(bool b) {
14             if (b) {
15                 *cell |= (1 << index);
16             } else {
17                 *cell &= ~(1 << index);
18             }
19             return *this;
20         }
21
22         operator bool() const { return *cell & (1 << index); }
23 };

```

```

24
25 public:
26     BitReference operator[](size_t index) {
27         return BitReference(arr_ + index / 8, index % 8);
28     }
29     // ...
30 };
31
32 int main() {
33     Vector<bool> v(10);
34     v[5] = true;
35 }

```

Это пример ситуации, когда rvalue можно и нужно что-то присваивать!

Правила инвалидации итераторов, указателей и ссылок для vector

Iterator invalidation

Operations	Invalidated
All read only operations	Never.
swap, std::swap	end()
clear, operator=, assign	Always.
reserve, shrink_to_fit	If the vector changed capacity, all of them. If not, none.
erase	Erased elements and all elements after them (including end()).
push_back, emplace_back	If the vector changed capacity, all of them. If not, only end().
insert, emplace	If the vector changed capacity, all of them. If not, only those at or after the insertion point (including end()).
resize	If the vector changed capacity, all of them. If not, only end() and any elements erased.
pop_back	The element erased and end().

Указатели и ссылки на элементы вектора инвалидируются в тех же случаях, когда инвалидируются соответствующие итераторы.

27 Внутреннее устройство std::deque

Разница между deque и vector: методы deque, отсутствующие у vector; методы vector, отсутствующие у deque. Правила инвалидации итераторов, указателей и ссылок для deque.

Как устроен deque изнутри? Идея реализации основных методов.

Как устроены итераторы deque?

Разница между deque и vector: методы deque, отсутствующие у vector

- ▷ pop_front
- ▷ push_front
- ▷ emplace_front

Методы vector, отсутствующие у deque

- ▷ reserve
- ▷ capacity
- ▷ shrink_to_fit

Правила инвалидации итераторов, указателей и ссылок для deque

Очень важное отличие std::deque от std::vector: в deque, в отличие от вектора, указатели и ссылки на элементы не инвалидируются от push_back, resize и т.д. Указатели и ссылки на элементы инвалидируются только если объект был удалён из дека.

Iterator invalidation

This section is incomplete
Reason: There are still a few inaccuracies in this section, refer to individual member function pages for more detail

Operations	Invalidated
All read only operations.	Never.
swap, std::swap	The past-the-end iterator may be invalidated (implementation defined).
shrink_to_fit, clear, insert, emplace, push_front, push_back, emplace_front, emplace_back	Always.
erase	If erasing at begin - only erased elements. If erasing at end - only erased elements and the past-the-end iterator. Otherwise - all iterators are invalidated. It is unspecified when the past-the-end iterator is invalidated.(until C++11) The past-the-end iterator is also invalidated unless the erased elements are at the beginning of the container and the last element is not erased.(since C++11)
resize	If the new size is smaller than the old one - only erased elements and the past-the-end iterator. If the new size is bigger than the old one - all iterators are invalidated. Otherwise - none iterators are invalidated.
pop_front, pop_back	To the element erased. The past-the-end iterator may be invalidated (implementation defined)(until C++11) is also invalidated.(since C++11)

Invalidation notes

- When inserting at either end of the deque, references are not invalidated by insert and emplace.
- push_front, push_back, emplace_front and emplace_back do not invalidate any references to elements of the deque.
- When erasing at either end of the deque, references to non-erased elements are not invalidated by erase, pop_front and pop_back.
- A call to resize with a smaller size does not invalidate any references to non-erased elements.
- A call to resize with a bigger size does not invalidate any references to elements of the deque.

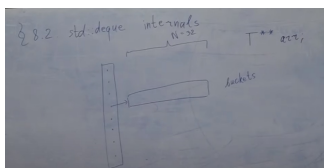
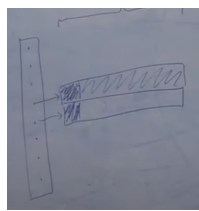
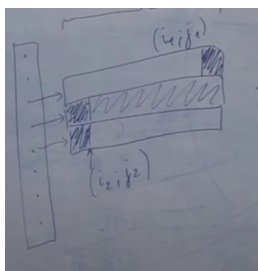
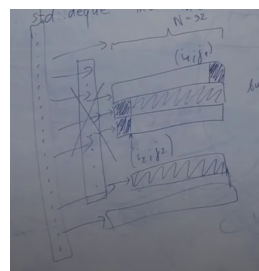


Рис. 1:

Рис. 2: После
push_backРис. 3: После
push_frontРис. 4: После ре-
аллокации

Как устроен deque изнутри? Идея реализации основных методов

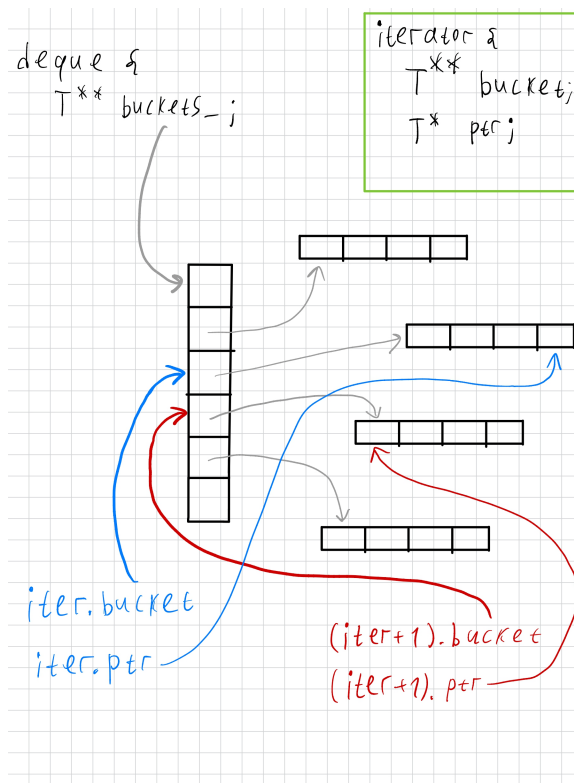
Реализуем дек следующим образом: заведём массив указателей, где каждый указатель указывает на соответствующий «bucket», массив фиксированного размера, где уже хранятся элементы дека. Изначально все указатели во внешнем массиве равны `nullptr`, т.е. есть только внешний массив и нет бакетов. При этом мы всегда храним два индекса, где находится начало дека и два индекса, где находится конец дека. Если нам нужно сделать `push_front`, то мы используем `placement-new` перед первым элементом и сдвигаем индексы начала дека. Если мы уже в самом начале бакета, то выделим бакет новый бакет перед первым и начнём заполнять его с конца. С `push_back` аналогично. Почему при такой устройстве дека указатели и ссылки на элементы не будут инвалидироваться? Дело в том, что реаллоцируется только внешний массив, а бакеты, где лежат объекты, не реаллоцируются, соответственно указатели и ссылки на элементы остаются валидными.

Чтобы удалить элемент из начала/конца дека, вызовем деструктор для первого/последнего элемента, а затем изменим координаты начала/конца.

Доступ по квадратным скобкам мы обеспечиваем за счёт того, что размер бакетов фиксирован, а значит, используя этот размер, координату начала и запрашиваемый индекс, можно вычислить координату искомого элемента.

Как устроены итераторы deque?

Мы хотим, чтобы разыменование итератора работало как можно быстрее, то есть требовало как можно меньше разыменований указателей. Поэтому в итераторе будем хранить указатель на элемент, на который сейчас указывает итератор. При этом нам нужно уметь инкрементировать итератор и, более того, прибавлять к нему целое число (мы хотим поддерживать `RandomAccessIterator`). Соответственно придётся переходить от одного бакета к следующему. Поэтому нам понадобится ещё одно поле — указатель на элемент внешнего массива, то есть указатель на указатель на начало бакета, поле типа `T**`. Тогда, чтобы перейти к следующему бакету, нужно будет просто инкрементировать указатель на бакет, а указатель на значение переставить на начало нового бакета.



Итого, если `ptr != *bucket + kBucketSize - 1`, то при инкременте итератора просто инкрементируем `ptr`. Если же `ptr == *bucket + kBucketSize - 1`, то `++bucket, ptr = *bucket;` С декрементом аналогично.

Поддержка арифметики указателей базируется на той же идее: индекс элемента в бакете $= ptr - *bucket$, поэтому из сдвига мы можем рассчитать новый номер элемента внутри бакета (`new_index`) и насколько должен измениться номер бакета (`bucket_shift`), тогда `bucket += bucket_shift, ptr = *bucket + new_index;`

Для ускорения работы инкремента за счёт увеличения размера итератора можно сделать следующее: можно добавить ещё одно поле, назовём его `end_of_bucket`, в котором будем хранить указатель на конец текущего бакета. Тогда проверка упростится до `ptr != end_of_bucket`, то есть мы избавились от разыменования в инкременте

Обратим внимание, что итераторы дека инвалидируются при добавлениях, так как при добавлении может произойти реаллокация внешнего массива, вследствие чего «внешние» указатели в итераторах станут недействительными.

списка, а перед фейковой нодой — последняя. Получается закольцованный список. При этом она естественным образом выполняет функцию end'a.

Осталось придумать как лучше всего реализовать это. Заведём структуру `BaseNode`, представителями которого будут все ноды, и фейковая, и обычные:

```
1 struct BaseNode {
2     BaseNode* prev;
3     BaseNode* next;
4 };
```

А теперь структуру для обычных нод (то есть хранящих значение)

```
1 struct Node : BaseNode {
2     T value;
3 };
```

Специфичные для list'a методы: splice, sort, merge, reverse, их асимптотики, идеи реализации

▷ sort()

Если вызвать обычный `std::sort` от листа, т.е. `std::sort(list.begin(), list.end())`, то получим СЕ, так как `std::sort` принимает `RandomAccessIterator`, а мы передали `Bidirectional`. Соответственно `std::list::sort` — это какая-то сортировка, обходящаяся `BidirectionalIterator`, например merge sort (merge sort достаточно forward iterator).

▷ merge(list& other)

«Сливает» два отсортированных списка. Работает за $O(n)$

▷ splice(const_iterator pos, list& other, const_iterator first, const_iterator last)

Вырезает кусок из одного листа и вставляет в другой (или в другое место этого)

Для этого достаточно перенаправить указатели, поэтому если вставка производится в тот же лист, то эта функция работает за $O(1)$. Но если мы переносим отрезок одного списка в другой, то нужно обновить их размеры, а для этого нужно знать сколько элементов мы переносим. Нужно подсчитать, сколько их, поэтому в этом случае splice работает за $O(\text{from} - \text{to})$

▷ reverse()

Меняет порядок элементов в контейнере на обратный. Никакие ссылки или итераторы не инвалидируются. Работает за $O(n)$

Класс `std::forward_list`, его отличия от обычного list

В то время как `std::list` — это двусвязный список, `std::forward_list` — это односвязным список

У него `ForwardIterator`

В нём нет `push_back` и `pop_back`, есть только `push_front` и `pop_front`.

У него нет метода `size`

Вместо `insert` у него `insert_after`, что логично, так как для вставки на переданный итератор (то есть перед находящемся там сейчас элементом) потребовался бы указатель в обратную сторону, так как нужно обновить указатель у предыдущего элемента. А после итератора можно вставить обойдясь тем, что есть.

29 Внутреннее устройство std::map

Класс `std::pair` и функция `std::make_pair`.

Внутренние типы `Node`, `ValueType`.

Поля класса `map`.

Реализация итераторов, алгоритмическая сложность инкремента итератора, прохода итератором по всему `map`.

Реализация методов `begin()` и `end()`, фейковая вершина.

Поддержка нестандартных компараторов в `map`.

Особенности работы оператора `[]`, отличие от `at()`.

Методы `find`, `insert`, `erase` с различными сигнатурами.

Правила инвалидации итераторов, указателей и ссылок для `map`.

Классы `std::set`, `std::multimap` и `std::multiset`, их предназначение, отличия от `std::map`.

Класс `std::pair` и функция `std::make_pair`

Класс `std::pair<T, U>` хранит два поля: `first` и `second`, типов `T` и `U` соответственно. Шаблонная функция `std::make_pair(a, b)` конструирует пару так, чтобы `a` было `first`, а `b` было `second` и возвращает эту пару.

Реализация методов `begin()` и `end()`, фейковая вершина

Фейковая вершина будет являться корнем дерева. Остальное дерево является левым сыном фейковой вершины, а правого сына у фейковой вершины нет. Пусть `begin` — это самый левый лист, а `end` — это фейковая вершина, тогда обход от `begin` до `end` обойдёт всё дерево

Внутренние типы `Node`, `ValueType`

```
1 struct BaseNode {
2     BaseNode* left;
3     BaseNode* right;
4     BaseNode* parent;
5     bool red;
6 };
```

Так как `BaseNode` хранит указатели, адрес `BaseNode` всегда выровнен по 8, а значит последние 3 байта в `parent` — всегда 0. Тогда можно потратить их с пользой и хранить в одном из этих бит красная вершина или нет. Тогда мы сможем избавиться от поля типа `bool` и сохранить 1 байт на каждую ноду.

```
1 struct BaseNode {
2     BaseNode* left;
3     BaseNode* right;
4     BaseNode* parent;
5 };
6
7 struct Node : BaseNode {
8     std::pair<const Key, Value> kv;
9     bool red;
10 };
```

Мы храним в ноде пару, потому что при разыменовании итератора надо возвращать что-то, в чём есть и ключ, и значение, а пара подходит для этой задачи.

Первый элемент пары — это `const Key`, так как без `const` можно было бы поменять по итератору ключ мапы, что нарушило бы её упорядоченность и привело бы к UB.

In progress...

Поля класса `map`

```
1 template <typename Key, typename Value, typename Compare = std::less<Key>>
2 class map {
3     BaseNode* fakeNode;
4     BaseNode* begin;
5     size_t sz;
6     Compare comp;
7     //...
8 }
```

Реализация итераторов, алгоритмическая сложность инкремента итератора, прохода итератором по всему `map`

Алгоритмическая сложность инкремента итератора — $O(\log n)$, так как в некоторых случаях нужно полностью подняться по дереву, то есть потратить $O(\text{высоты дерева})$ операций, а высота сбалансированного дерева — логарифм от числа вершин в нём

В то же время, алгоритмическая сложность прохода итератором по всему `map` — это $O(n)$. Понять это можно, если заметить, что каждую вершину мы посещаем не более трёх раз

In progress...

Поддержка нестандартных компараторов в `map`

Последний шаблонный аргумент класса `std::map` — это тип компаратора, по умолчанию он равен `std::less<Key>`. Компаратор — функциональный объект, у которого есть оператор круглые скобки от двух ключей. `std::map` сравнивает ключи всегда только с помощью компаратора, соответственно содержимое контейнера поддерживается отсортированным относительно компаратора. Компаратор должен удовлетворять свойствам строгого отношения порядка, иначе UB

Особенности работы оператора `[]`, отличие от `at()`

Если `at()` не находит нужный ключ, он кидает исключение, а `operator[]` если не находит запрошенный ключ, создаёт пару с таким ключом (значение создаётся конструктором по умолчанию). Поэтому квадратные скобки не помечены как `const`, то есть

квадратные скобки нельзя использовать на константном `map`. Соответственно, в константном `map` искать с помощью `at`, а лучше `find`.

Методы `find`, `insert`, `erase` с различными сигнатурами

Пример использования `find`: Пусть мы хотим проверить, есть ли в `map` ключ, и если есть, присвоить туда некоторое значение

```
1 if (m.count(1)) { // since C++ 20 we can use m.contains(1)
2     m[1] = 2;
3 }
```

Код выше не очень эффективен, так как использует два спуска по дереву: первый, чтобы проверить наличие ключа, второй, чтобы обновить значение.

Вместо этого лучше использовать `find`:

```

1 if (auto it = m.find(1); it != m.end()) { // if with initialization, since
    C++ 17
2     it->second = 2;
3 }

```

std::map<Key,T,Compare,Allocator>::find

iterator find(const Key& key);	(1)
const_iterator find(const Key& key) const;	(2)
template< class K > iterator find(const K& x);	(3) (since C++14)
template< class K > const_iterator find(const K& x) const;	(4) (since C++14)

1,2) Finds an element with key equivalent to `key`.

3,4) Finds an element with key that compares *equivalent* to the value `x`. This overload participates in overload resolution only if the qualified-id `Compare::is_transparent` is valid and denotes a type. It allows calling this function without constructing an instance of `Key`.

std::map<Key,T,Compare,Allocator>::erase

iterator erase(iterator pos);	(1)
iterator erase(const_iterator pos);	(2) (since C++11)
iterator erase(iterator first, iterator last);	(3) (until C++11)
iterator erase(const_iterator first, const_iterator last);	(3) (since C++11)
size_type erase(const Key& key);	(4)
template< class K > size_type erase(K&& x);	(5) (since C++23)

Removes specified elements from the container.

1,2) Removes the element at `pos`.

3) Removes the elements in the range `[ first, last )`, which must be a valid range in `*this`.

4) Removes the element (if one exists) with the key equivalent to `key`.

5) Removes all elements with key that compares *equivalent* to the value `x`. This overload participates in overload resolution only if the qualified-id `Compare::is_transparent` is valid and denotes a type, and neither `iterator` nor `const_iterator` is implicitly convertible from `K`. It allows calling this function without constructing an instance of `Key`.

References and iterators to the erased elements are invalidated. Other references and iterators are not affected.

The iterator `pos` must be valid and dereferenceable. Thus the `end()` iterator (which is valid, but is not dereferenceable) cannot be used as a value for `pos`.

std::map<Key,T,Compare,Allocator>::insert

std::pair<iterator, bool> insert(const value_type& value);	(1)
template< class P > std::pair<iterator, bool> insert(P&& value);	(2) (since C++11)
std::pair<iterator, bool> insert(value_type&& value);	(3) (since C++17)
iterator insert(iterator pos, const value_type& value);	(4) (until C++11)
iterator insert(const_iterator pos, const value_type& value);	(4) (since C++11)
template< class P > iterator insert(const_iterator pos, P&& value);	(5) (since C++11)
iterator insert(const_iterator pos, value_type&& value);	(6) (since C++17)
template< class InputIt > void insert(InputIt first, InputIt last);	(7)
void insert(std::initializer_list<value_type> ilist);	(8) (since C++11)
insert_return_type insert(node_type&& nh);	(9) (since C++17)
iterator insert(const_iterator pos, node_type&& nh);	(10) (since C++17)

In progress...

Правила инвалидации итераторов, указателей и ссылок для map

In progress...

Классы `std::set`, `std::multimap` и `std::multiset`, их предназначение, отличия от `std::map`

In progress...

30 Аллокаторы

Мотивировка использования аллокаторов. Понятие аллокатора. Класс `std::allocator`, его реализация. Исправление реализации `vector::reserve` с учетом аллокатора. Структура `allocator_traits`. Использование методов `allocator_traits` вместо соответствующих методов аллокаторов.

Мотивация:

1. хотим завести контейнер с нодами и добавляем в него миллион элементов по одному, это каждый раз вызов `new`, а `new` долгая операция, нам это не нравится;
2. хотим создавать контейнеры на стеке, на конкретное количество элементов.

Поэтому неплохо было бы просить контейнер брать память из другого места.

Аллокатор — промежуточный уровень абстракции между контейнером и оператором `new`.

Container -> Allocator -> operator new -> malloc -> OS

Пример: создаем класс, в котором конструктор вызывает `new` сразу на очень много элементов, а потом отдает нам указатель на свободный кусок.

```

1 template <typename T>
2 struct allocator {
3     T* allocate(size_t count) {
4         return reinterpret_cast<T*>(new char[count * sizeof(T)]);
5     }
6     /*
7     template <typename U, typename... Args>
8     void construct(U* ptr, const Args&... args) {
9         new (ptr) U(args...);
10    }
11    void destroy(T* ptr) {
12        ptr->~T();
13    }
14    */
15    //there are no such methods, as they are in allocator_traits
16    void deallocate(T* ptr, size_t) {
17        delete[] reinterpret_cast<char*>(ptr);
18    }
19 }
```

Это stateless allocator, потому что он не хранит никаких состояний, а является просто оберткой над `new` и `delete`.

```

1 // using AllocTraits = std::allocator_traits
2 void reserve(size_t newcap) {
3     if (newcap <= cap) {
4         return;
5     }
6
7     T* newarr = alloc.allocate(newcap); // AllocTraits::allocate(alloc,
newcap);
8     size_t i = 0;
9     try {
10        for (; i < sz; ++i) {
11            alloc.construct(newarr + i, arr[i]); // AllocTraits::construct(
alloc, newarr + i, arr[i])

```

```

12     }
13 } catch (...) {
14     for (size_t j = 0; j < i; ++i) {
15         alloc.destroy(arr + i); // AllocTraits::destroy(alloc, newarr +
16     j)
17     }
18     alloc.deallocate(newarr, newcap); // AllocTraits::deallocate(alloc,
19 newarr, newcap)
20 }
21 for (size_t i = 0; i < sz; ++i) {
22     alloc.destroy(arr + i); // AllocTraits::destroy(alloc, arr + i)
23 }
24 alloc.deallocate(arr, cap); // AllocTraits::deallocate(alloc, arr, cap)
25
26 cap = newcap;
27 arr = newarr;
28 }

```

Allocator_traits: вынесем во внешний класс общие аспекты аллокаторов

```

1 template <typename Alloc>
2 struct allocator_traits {
3     using value_type = typename Alloc::value_type;
4
5     static value_type* allocate(Alloc& alloc, size_t count) {
6         return alloc.allocate(count);
7     }
8
9     static void deallocate(Alloc& alloc, T* ptr, size_t count) {
10         alloc.deallocate(ptr, count);
11     }
12
13     template <typename U, typename... Args>
14     static void construct(Alloc& alloc, U* ptr, const Args&... args) {
15         if constexpr (/* Alloc has method construct(U*, const Args&...) */)
16         {
17             alloc.construct(ptr, args...);
18         } else {
19             new(ptr) U(args...);
20         }
21     }
22
23     template <typename U, typename... Args>
24     static void destroy(Alloc& alloc, U* ptr) {
25         if constexpr (/* Alloc has method destroy(U*) */) {
26             alloc.destroy(ptr);
27         } else {
28             ptr->~U();
29         }
30     }
31
32     alloc.allocate(newcap) -> AllocTraits::allocate(alloc, newcap);
33     alloc.construct(newarr + i, arr[i]) -> AllocTraits::construct(alloc, newarr
34 + i, arr[i])
35     alloc.destroy(arr + i) -> AllocTraits::destroy(alloc, newarr + j)
36     alloc.deallocate(newarr, newcap) -> AllocTraits::deallocate(alloc, newarr,
37 newcap)

```

31 Формальное определение lvalue и rvalue

Определение вида value для всевозможных выражений. Пример rvalue, которому можно присваивать, и lvalue, которому нельзя присваивать. Новый вид ссылок: rvalue reference. Ссылочные квалификаторы (ref-qualifiers) для методов классов, пример их применения. Что происходит при комбинации const- и ref-квалификаторов?

32 Rvalue-ссылки

Сходства и различия между обычными ссылками и rvalue-ссылками. Особенности инициализации rvalue-ссылок, разрешенные и запрещенные присваивания между различными видами ссылок с разными категориями value, а также нессылочными объектами. Как это все сочетается с константными типами?

33 Проблема perfect forwarding

Понятие универсальных ссылок, отличие их от rvalue-ссылок. Особенности типа `auto&&`. Правила вывода типа шаблонов в случае универсальных ссылок. Правила сворачивания ссылок (reference collapsing). Функция `std::forward` и ее применение. Решение проблем с `emplace_back` и `construct`. Реализация функции `std::move`. Объяснение, почему принимаемый и возвращаемый типы именно такие.

34 Ключевое слово `decltype` и конструкция `decltype(auto)`

Использование `decltype` от переменных, простые примеры. Особенности вывода типа при `decltype`. Особенности поведения `decltype` от выражений при различных видах `value`. Конструкция `decltype(auto)`, ее действие и пример использования.

35 Классы `unique_ptr` и `shared_ptr`

Идея реализации `shared_ptr` (без нестандартного делетера и аллокатора). Реализация конструкторов, операторов присваивания и деструктора. Использование нестандартного делетера в `unique_ptr`, реализация. Функции `make_unique` и `make_shared`. Какие проблемы они решают? Реализация этих функций, исправление реализации `shared_ptr` для их поддержки.

`unique_ptr`: Класс, который хранит указатель на объект и запрещает разделять владение им. Если вдруг объект, который хранил `unique_ptr`, сломался, то `unique_ptr` сам очистит память.

Реализация: Единственное поле, которое нам нужно (помимо делетера и аллокатора) - это указатель на объект. Класс не должен уметь копироваться и присваиваться, но должен уметь муваться; в деструкторе нужно вызвать деструктор объекта под указателем и освободить память. Такая реализация избавляет нас от огромного количества геморроя, который появится дальше.

`shared_ptr`: То же самое, но теперь у нас нет уникальности, т.е. кто угодно и в любых количествах может копировать и мувать наш умный указатель.

Реализация: Помимо указателя на объект, храним количество `shared_ptr`, ссылающихся на указатель. В копи-конструкторе (и операторе присваивания) инкрементируем его, в деструкторе - декрементируем.

Идея реализации `shared_ptr`: Храним структуру, в которой лежит указатель и эти самых счётчики (ну и делетер с аллокатором). Самый приколом этой структуры в том, что мы вызываем оператор `new` всего один раз (а мы хотим вызывать его как можно реже). Когда счётчик обнулится - поздравляю, теперь никто не хранит указатель на этот объект, значит, можно с чистой совестью его удалять (проверку на это делаем в деструкторе).

Функции `make_unique` и `make_shared`: Допустим, мы хотим создать объект в динамической памяти и сразу определить для него (`unique`) `shared_ptr` - но погодите: тогда мы вызовем оператор `new` дважды - создавая объект и умный указатель на него. А хотим вызвать один раз! Решение есть: давайте создадим ещё один `ControlBlock`, и в него положим, помимо всего прочего, сам объект. Таким образом, оператор `new` вызовется всего один раз, а ещё нам не придётся использовать его непосредственно.

Использование нестандартного делетера в `unique_ptr`: а вот и самое интересное. Допустим, что наш объект - это что-то нетривиальное и создаётся он как-то по-особому, а значит, и удаляется тоже. Настолько, что нам для этого нужен специальный функтор (т.е. *какая-то штука, у которой есть `operator()`*). Самый простой пример - логгер: мы хотим не просто удалить его, а ещё и остановить работу.

Возникает вопрос, а почему бы не сделать это всё в деструкторе? - Как правило, мы так и будем делать, но если вдруг это не наш класс, а какая-то сторонняя библиотека, то нельзя просто так взять и переопределить деструктор. Поэтому руки в ноги и писать делетер. Второй вопрос: если есть нестандартный делетер, то должен быть и стандартный? - Да, такой есть, и называется он `std::default_delete`. В целом - можно не задумываться об

его устройстве.

weak_ptr, кто это и зачем он нужен: представим ситуацию - у нас есть связный список, написанный на shared_ptr, т.е. указатель на следующую вершину реализован так:

```
1 std::shared_ptr<Node> next;
```

...и пусть у нас образовался цикл из таких вершинок. Тогда утверждается, что мы никогда не вызовем делетер, а значит, нужно что-то менять. В частности, нужно завести структуру weak_ptr, которая делает примерно то же самое, но не учитывается в подсчёте. Грубо говоря: если на объект не указывает ни один shared_ptr, а только объекты weak_ptr, то мы с чистой совестью вызываем делетер (но не удаляем ControlBlock).

А как его написать - расскажу в следующей серии. Ставьте лайки, подписывайтесь на канал.

36 Лямбда-функции и замыкания

Применение слова `mutable` в лямбда-функциях. Захват с инициализацией (since C++14). Пример захвата `move-only` объекта посредством `std::move`. Пример захвата по константной ссылке посредством `std::as_const`. Пример битой ссылки в случае неаккуратного захвата. Захват по умолчанию и проблемы, которые он потенциально порождает. Особенности захвата `this` и полей класса. Использование слова `auto` в лямбда-функциях.

Как уже известно, мы можем принимать в лямбда функцию переменные по ссылке или по значению, но в то же время возникает вопрос, а можем ли мы менять их внутри функции?

Объекты захваченные по ссылке можно менять в любом случае. В свою очередь объекты захваченные по значению менять нельзя, в виду того, что `operator()` у замыкания это константный метод, а значит на для него на поля навешен `const`. При этом навешивание `const` на ссылку не оказывается никакого эффекта, а поэтому их можно менять.

Чтобы избежать данной проблемы с объектами захваченными по значению можно использовать ключевое слово `mutable`, оно даст нам изменять объекты даже при условии константных методов.

```
1 auto cmp = [c](int a, int b) mutable {
2     ++c;
3 };
```

Еще одним способом захвата переменной будет захват с инициализацией, позволяющий нам сохранить измененную версию переменной.

```
1 auto cmp = [c = c + 2](int a, int b) {};
```

С помощью этого инструмента можно захватывать объекты через `move` и захватывать объекты по константной ссылке.

```
1 auto cmp_move = [c = std::move(c)](int a, int b) {};
```

```
2
```

```
3 auto cmp_const_reference = [c = std::as_const(c)](int a, int b) {};
```

Логика работы ссылок при захвате их в лямбда функции во многом похожа на поведение обычных ссылок, в частности можно отметить пример возврата битой ссылки.

```
1 auto f() {
2     int tmp_value = 5;
3     return [&tmp_value]() { std::cout << tmp_value; };
4 }
5
```

```
6 int main() {
7     auto lambda = f();
8     lambda(); // UB
9 }
```

Также существует возможность захватить сразу все локальные переменные в лямбда функцию. Это также можно делать по ссылке или по значению, с помощью [&] или [=] соответственно. Кроме того мы можем дополнительно определять некоторые уточнения, например: захватить все локальные переменные по значению, а какую-то по ссылке.

```
1 int c = 5;
2
3 auto cmp_all_reference = [&](int a, int b) {};
4
5 auto cmp_all_copy = [=, &c](int a, int b) {};
```

Подобный синтаксис считается не желательным и его лучше не использовать, так как это может привести к некоторым распространенным ошибкам. В частности пример битой ссылки, копирование "тяжелых" объектов и другие ошибки.

При работе с лямбда функциями в классах стоит помнить, что его поля не являются локальными переменными, а потому не будут захвачены при захвате по умолчанию, что также может привести к UB в случае если объект удалится раньше, чем мы используем лямбда функцию, возвращенную из него.

```
1 struct S {
2     int t;
3     void f() {
4         return [=](int x) { std::cout << x + t; };
5     }
6 }
7
8 int main() {
9     S* s = new S();
10    auto lambda = s->f();
11    delete s;
12
13    lambda(5); // UB
14 }
```

В примере выше на самом деле мы не завытываем t, а захватываем this, с которым работать после его удаления невозможно.

В свою очередь переменные класса будут доступны из тела лямбды.

Также мы можем самостоятельно захватить this с помощью уже рассмотренного синтаксиса, а благодаря захвату с инициализацией — получить и явную копию нашего объекта.

В объявлении и теле лямбда функций можно использовать ключевое слово auto по аналогии с его использованием в объявлении и теле обычных функций.

```
1 auto f = [](auto a, auto b) {
2     return a + b;
3 };
```

Основным отличием при этом будет то, что на самом деле шаблонным становится operator(), а не сам класс замыкания.

37 Шаблонное метапрограммирование и SFINAE

Структуры `std::integral_constant`, `std::bool_constant`, типы `true_type` и `false_type`, структура `std::type_identity`, структуры `std::conjunction`, `std::disjunction` (все - с реализацией). Примеры простых метафункций с реализацией. Идиома SFINAE. Пример выбора между двумя функциями с помощью SFINAE. Структура `std::enable_if`, ее реализация и примеры применения.

std::integral_constant — структура которая позволяет определить константу для метапрограммирования

```
1 template <typename T, T x>
2 struct integral_constant {
3     static constexpr T value = x;
4 }
```

std::bool_constant — аналогично `std::integral_constant` для `bool`.

```
1 template <bool b>
2 using bool_constant = integral_constant<bool, b>;
```

std::true_type — структуры константных `true` и `false`.

```
1 using true_type = bool_constant<true>;
2 using false_type = bool_constant<false>;
```

std::type_identity — по сути структура для типа дающая этот же тип.

```
1 template <typename T>
2 struct type_identity {
3     using type = T;
4 }
5
6 template <typename T>
7 using type_identity_t = type_identity<T>::type;
```

std::conjunction — структура определяющая конъюнкцию. Ее реализация могла бы быть тривиальным упражнением на использование пакетов, но к ней выдвигается дополнительное требование, а именно - мы не хотим производить инстанцирование если уже может вычислить значение `conjunction`. То есть если в какой то момент мы находим шаблонный аргумент с значением `false`, то мы не должны производить инстанцирование последующих аргументов. Этого можно добиться с помощью `std::conditional`.

std::disjunction — аналогичным образом объявляет дизъюнкцию.

```
1 template <typename...>
2 struct conjunction : std::true_type {}
3
4 template <typename Head, typename... Tail>
5 struct conjunction<Head, Tail...> :
6     std::conditional_t<bool(Head::value), conjunction<Tail...>, Head> {};
```

Идиома *SFINAE* (*substitution failure is not an error*) — идиома означающая, что если при инстанцировании объявления шаблонной функции мы получаем некорректный тип или значение, то это не будет воспринято как ошибка компиляции, а данный кандидат на перегрузку просто отметится как невалидный.

```

1 template <typename T>
2 auto f(T) -> typename T::value_type {
3     std::cout << 1 << '\n';
4     return {};
5 }
6
7 // C-style variadic function
8 void f(...) {
9     std::cout << 2 << '\n';
10 }
11
12 int main() {
13     std::vector<int> v;
14     f(v); // prints 1, because first is acceptable
15     f(0); // prints 2, because first is incorrect and SFINAE works
16 }

```

`std::enable_if` — структура позволяющая нам осуществлять выбор кандидата на перегрузку на основе заданного условия. Можно заметить, что это нельзя сделать с помощью простого `conditional_t`, ведь объявление функции все равно придется инстанцировать, а значит мы не сможем задать там некоторые зависящие от шаблонного аргумента условия. В свою очередь `std::enable_if` выдает некорректный тип, в случае если условие неверно, позволяя тем самым сработать SFINAE и убрать эту перегрузку из рассмотрения. Аргумент `T` нужен для того, чтобы не происходило переопределения при попытке задать несколько функций. Вместо того чтобы задавать тип по умолчанию мы зададим переменную по умолчанию, а из-за формального различия типов переменных мы сможем корректно задать перегрузку. При этом очевидно, значение самой переменной и ее имя значения не имеют.

```

1 template <bool B, typename T>
2 struct enable_if {};
3
4 template <typename T>
5 struct enable_if<true, T> {
6     using type = T;
7 }
8
9 template <bool B, typename T>
10 using enable_if_t = enable_if<B, T>::type;
11
12 template<typename T, enable_if_t<some_condition<T>, bool> = true>
13 auto f(T) {}
14
15 template <typename T, enable_if_t<!some_condition<T>, bool> = false>
16 auto f(T) {}

```

38 Ограничения, ключевое слово `requires`

Конструкция `requires` для наложения ограничений на принимаемые типы. Полный и сокращенный синтаксис `requires`. Оператор `requires` и пример его использования. Четыре вида ограничений: простые, составные, ограничения на типы и вложенные ограничения (с примерами).

requires — ключевое слово которое позволяет нам задавать ограничения на шаблонные аргументы. В случае если аргумент не соответствует требованиям будет СЕ. Стоит отметить, что на нешаблонные функции `requires` работать не будет.

При сокращенном синтаксисе мы можем проверить какое-то простое условие.

```
1 template <typename T, size_t N>
2 requires (N <= 1000 && N >= 100)
3 struct array {};
4
5 template <typename T>
6 requires std::is_integral_v(T)
7 void f(T) {}
```

При полной синтаксисе мы можем задать более сложные условия, как соответствие концепту, наличие типа и компилируемость выражения.

```
1 template <typename T>
2 requires requires(T a, T b) {
3     a + b;
4 }
5 T sum(T a, T b) {
6     return a + b;
7 }
```

Существует 4 вида возможных ограничений: 1. Простые ограничения задают элементарные условия на типы, такие как наличие конкретного оператора или метода.

```
1 template<typename T>
2 requires requires(T a, T b) {
3     { a + b };
4 }
5 T add(T a, T b) {
6     return a + b;
7 }
```

2. Составные ограничения комбинируют несколько условий с использованием логических операторов.

```
1 template<typename T>
2 requires requires(T a, T b) {
3     { a + b };
4 } && requires(T a, T b) {
5     { a * b };
6 }
7 T add_and_multiply(T a, T b) {
8     return (a + b) * b;
9 }
```

3. Ограничения на типы задают условия на сами типы, такие как соответствие определённому концепту или наличие определённых свойств.

```
1 template<typename T>
2 requires std::integral<T>
3 T multiply(T a, T b) {
4     return a * b;
5 }
```

4. Вложенные ограничения накладывают условия на шаблонные параметры на соответствие концепту. Это полезно для проверки условий в более сложных структурах.

```
1 template<typename Container>
2 requires requires(Container c) {
3     typename Container::value_type;
4     { c.begin() } -> std::input_iterator;
5     { c.end() } -> std::input_iterator;
6 }
7 auto sum(Container const& c) {
8     // something here
9 }
```

39 Класс std::function

Предназначение и примеры применения `std::function`. Использование `std::function` для обычных указателей на функции, для функциональных объектов, для замыканий.

`std::function` — класс позволяющий нам хранить функцию как объект. Благодаря нему мы можем, например, передать функцию как аргумент в другую функцию.

Мы можем создавать `std::function` от обычного указателя на функцию.

```
1 int apply(int x, const std::function<int(int)>& f) {
2     return f(x);
3 }
4
5 int square(int x) {
6     return x * x;
7 }
8
9 int main() {
10     std::function<int(int)> f = square;
11     std::cout << apply(5, f);
12 }
```

Также мы можем задавать `std::function` от объекта с определенным методом `operator()`. Такие объекты иногда называют функциональными.

```
1 struct AddOne {
2     int operator()(int x) {
3         return x + 1;
4     }
5 }
6
7 int main() {
8     std::function<int(int)> f = AddOne();
9     std::cout << f(6);
10 }
```

Еще мы можем создавать `std::function` от замыкания. Важно отметить что при этом особенности взаимодействия замыкания с захваченными объектами сохраняются и при передаче их в `std::function`. При этом при передаче по ссылке мы очевидно меняем тот же объект.

```
1 int c = 2;
2 auto mult = [&c](int x) {
3     ++c;
4     return x * c;
5 }
6 std::function<int(int)> f = mult;
7 std::cout << f(6);
```